Intelligence Is Not Authority

Verified Policy Decisions and Revocable Local Agent Authority

Jepson Taylor
jepson@veox.ai
10 November 2026

Abstract. What authority can remain after an agent is stopped or permission is withdrawn? We separate admission revocation from sink-fenced revocation and implement a local broker and cooperative tool service with distinct durable stores. Reservations precede dispatch; ambiguous outcomes retain their charges. Under the stated sink contract, effects after admission revocation are bounded by outstanding reserved charges, while a durable fence acknowledgment excludes subsequent effects under the withdrawn issuer epoch. A pinned Verus run verifies 17 obligations for the deterministic Rust policy core compiled into the broker; parsing, storage, normalization, session identity, and the kernel remain trusted. Independent trace checks, crash regressions, semantic positive controls, and Landlock/seccomp probes exercise this boundary. Ten abstract mechanism omissions and all 45 pairs have witnesses within a declared finite search language. Scripted CPU comparisons and native AgentDojo/Progent integration controls are reported separately from model experiments. All checked Linux GPUs were occupied; the planned Qwen3 checkpoint comparison is unrun, so no empirical claim about adversarial model utility or noninferiority is made. The result is a conditional authorization and revocation artifact, not verification of an entire runtime or an arbitrary external service.

1 The authority left after stopping

An agent process can stop while its authorized work continues elsewhere. A tool may have accepted a payment, queued a release, or begun a document update before the operator withdraws permission. Killing the proposer does not erase those requests. A timeout does not reveal whether they committed. Returning their budget to a replacement agent can authorize a second effect while the first remains possible. The relevant control question is therefore about outstanding authority, rather than the lifetime of the model process.

This paper asks what authority can remain after stopping or withdrawal, which mechanisms bound it, and what evidence those mechanisms support. The artifact uses synthetic payments, releases, and edits, all within local services. These three effect types deliberately require different observations: recipient and amount for payments, destination and actual contents for releases, and revision plus expected contents for edits. An authorization result concerns the encoded permission. A task-success result concerns the intended effect. Neither result establishes that the issuer had legitimate jurisdiction over the resource.

1.1 Contribution and claim conditions

We distinguish three proposed claims. First, a deterministic core makes its specified authorization decisions independently of proposer behavior. Its executable predicates have mechanized equivalence specifications, including positive permissions; conservation arguments quantify over arbitrary finite prefixes. The broker compiles this core as a Rust dependency. This verifies particular decisions and algebraic lemmas, not the parser, database program, or whole runtime. Their correspondence to the arguments supplied to the core is an explicit trusted premise.

Second, revocation has two meanings. Admission revocation stops issuance of new dispatch permits. Previously issued permits may still reach the sink and cause effects. Their additional charge is bounded by outstanding reservations under the cooperative sink contract. Fenced revocation adds a durable sink barrier, which cancels outstanding old-epoch work or waits behind work that already commits. Its acknowledgment marks a stronger boundary: subsequent old-authority effects are excluded. We give the protocol argument, verify its decision predicates, and exercise separate-process fault and concurrency tests.

Third, useful work under evaluated model attacks would require comparative empirical evidence. That claim remains contingent. The release includes CPU positive controls, descriptive controller controls, pinned public-benchmark integration checks, and a predeclared model/inference manifest. It does not include completed local-model campaigns. Host observations show no idle GPU on the verified Linux hosts. No inference about attack success, checkpoint capability, or usefulness non-

inferiority follows from those missing results.

1.2 Relation to existing permission controls

CaMeL separates control and data flow and uses explicit capabilities for agent tool execution [37]. Progent checks symbolic tool-argument policies and controls their evolution [38]. Permission checking by itself is therefore not a new contribution. Fault-tolerant budget conservation already addresses reservations, delayed effects, and authoritative settlement under declared gateway assumptions [39]. Our focus is the connection between an executable policy core, a small separate-service implementation, and a precise local revocation acknowledgment.

The sink contract is central to this comparison. LIMBO studies duplicate side effects under ambiguous replies and differing service contracts [40]. We do not solve arbitrary external exactly-once execution. The supported sink must atomically bind an operation identity to its normalized effect, deduplicate it, and apply any effect in that same transaction. A deliberately non-idempotent service is retained as an unsupported-contract negative control. Its failure is not counted as a security failure of a contract it never implemented.

These distinctions also constrain interpretation of verification. Reference monitor principles motivate a small mediation boundary [14, 15]. Increasing the number of checked synthetic states does not close a missing operating-system path, validate a normalization routine, or make an incomplete harm specification adequate. The v1 finite model and synthetic learning studies are preserved in appendices so their evidence remains inspectable without carrying the main revocation claim.

1.3 Threat model

The proposer can choose arbitrary messages, malformed requests, altered payloads, retries, timing, and coordination among a finite number of actors. It may try to impersonate an issuer or access databases and the sink directly. The policy must not depend on the proposer reporting its intent, being helpful, or recognizing revocation. Actors share one budget; copying an actor does not copy that budget. An arbitrary finite actor population means any number of proposers using the specified interface, not an unbounded resource-exhaustion guarantee for a shared host.

The trusted boundary contains the session launcher, administrator channels, normalization, the broker and sink storage adapters, the compiler and verifier, and the Linux kernel. Administrators provision exact approvals, withdraw issuers, and settle authoritative sink outcomes.

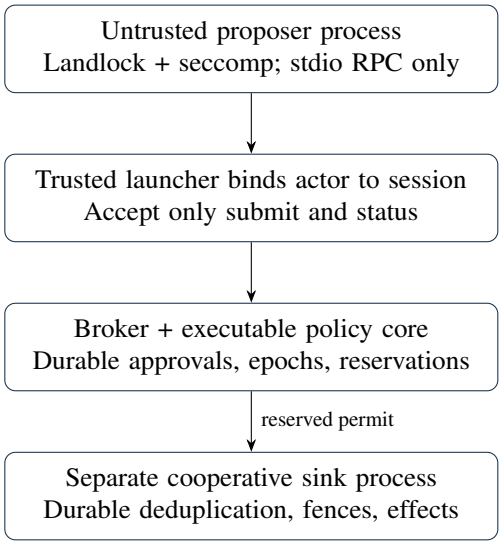


Figure 1: The proposer has no database or sink descriptor. Administrator operations use separate trusted channels. Each service owns a different durable store; the drawing describes the local prototype, not a distributed ledger.

The proposer receives none of those commands. Storage and service processes may crash; they must preserve committed state on recovery. We test process death, not corrupted disks, Byzantine databases, compromised administrators, kernel exploits, or malicious sinks. A sink that ignores deduplication or a fence invalidates the associated bound rather than becoming an adversarial case covered by it.

We treat scheduler and storage delays as availability costs. The system does not promise a hard deadline for revocation acknowledgment. If the sink cannot return authoritative completion or cancellation, the broker retains the charge and the strong acknowledgment remains unavailable. This conservative outcome is part of the protocol, not an experimental failure to be hidden.

2 Authorization protocol and process boundary

The v1 executor places policy, receipts, and protected effects in one SQLite transaction. It remains a useful atomic baseline. V2 separates the broker's accounting store from the sink's effect store and connects their owning processes through trusted JSONL channels. The sink cannot rely on an atomic transaction with the broker. The design therefore reserves before dispatch and treats ambiguous outcomes as outstanding authority.

2.1 Normalized requests and approvals

A proposal includes an immutable operation identifier, an issuer-epoch vector, an expiry, and a typed effect. The launcher supplies the actor identity. The normalized per-

mit binds all these fields. A transfer names its recipient and units; a release names its destination and actual contents; an edit names its document, expected revision, and replacement contents. Serialization of the typed permit supplies the byte sequence checked against the approved permit. Unknown proposal fields are rejected. JSON whitespace and field order are not alternate authorities because parsing precedes canonical typed serialization.

The local fixture has two issuers. Transfers require the operator; releases and edits require both operator and steward. The core decision supports arbitrary finite issuer vectors, while the application instantiates two. Approval for one identifier cannot be altered into approval for another payload. Reusing an identifier with changed contents or a different session actor is denied. A status response contains no newly released data and does not repeat an effect.

Approvals and issuer liveness are distinct. Each required issuer must have an approval bit, a live authorization, and a current epoch matching the request. Withdrawal advances that issuer's epoch and clears its approval bits. Renewal can make the issuer live again but does not restore those bits, reset the budget, or lower sink fence floors. Fresh work needs fresh approvals. A transfer that requires only the operator is not revoked merely by withdrawing the steward. Both independent withdrawal and required-grant conjunction are tested.

The prototype's scope predicate deliberately remains small. Transfers go to the synthetic approved recipient, releases contain the fixed synthetic record, and edits target a toy document. Exact approval can authorize a bad replacement document; task correctness is checked separately. There is no claim that these few scope rules implement a general data-governance policy or a real financial authorization system.

2.2 Reservation and dispatch

Let B be the budget, C charged authority, and $q > 0$ a request's maximum charge. Admission requires $q \leq B$ and $C \leq B - q$. The subtraction form avoids unsigned overflow. On acceptance the broker atomically persists the immutable permit and sets $C' = C + q$. Only then can the trusted launcher forward the permit to the sink. If the broker dies after commit but before replying, retry returns the same pending permit; it creates no second reservation.

Issuing a dispatch permit is the admission linearization point. A permit can be in transit when withdrawal occurs. The protocol therefore treats every issued but unresolved permit as outstanding, including one the sink has not yet observed. Admission revocation closes issuance of new permits; it cannot recall a byte sequence already handed to a transport. Recovering or retrying such an identity is not a newly minted authority. The sink still deduplicates or fences it.

The sink first durably accepts a permit as pending. Completion occurs in a transaction that checks the required issuer fence floors and, for an edit, the current object revision. It then applies the effect and records committed status atomically. A unique operation identity binds that status to the same permit. Duplicate completion requests retrieve terminal status and do not apply another effect. A stale edit is durably cancelled, allowing settlement without inventing a document update.

2.3 Settlement under uncertainty

A committed sink response settles the reservation as consumed. The charge stays in C; settlement does not replenish a cumulative budget. A durably cancelled sink identity supplies authoritative evidence of no effect and can release its reservation. Cancellation is terminal: a later delivery of the same identity cannot commit. Repeated settlement cannot refund twice because the broker allows settlement only from pending status.

No timeout duration appears in the core's release rule. Pending, unknown, EOF, and lost replies provide no no-effect certificate. In particular, absence from a sink lookup is insufficient: an older dispatch could still arrive. After a fence, the sink can record a cancellation tombstone for such a late permit. The broker may then release that reservation because the terminal identity and fence prevent its future effect. A transport failure before this evidence leaves the reservation stranded, conservatively reducing availability.

The local channels authenticate service provenance by construction: proposer children receive only their own stdio pipes. A remote deployment would need authenticated permits, replies, and fence acknowledgments as well as durable ordering across every sink. No cryptographic remote protocol is evaluated here. An administrator who fabricates settlement evidence is outside the threat model. The broker's settlement endpoint must never be exposed as proposer RPC.

2.4 Kernel-enforced proposer restrictions

The launcher creates the session and binds its actor identifier before processing a child message. It accepts only submission and status; extra actor fields and administrator commands are denied. Before executing the child it sets no-new-privileges, applies Landlock filesystem

restrictions, installs a seccomp filter, closes non-stdio descriptors, and clears the environment. Only runtime libraries and the chosen executable are readable. Broker and sink databases, credentials, and the control channels are outside the accessible filesystem and descriptor set.

The implementation requires Landlock ABI at least 4 and fails startup if it cannot install the rules. It does not require user namespaces, which were unavailable on this host. Landlock constrains filesystem access; seccomp denies socket creation and connection, cross-process memory inspection, tracing, and additional bypass-related syscalls. The architecture check rejects alternate syscall ABIs. These are complementary kernel mechanisms, not a proof of an entire sandbox [42, 43].

Native controls must demonstrate that the probes can succeed without confinement. The restricted runs then try database reads and writes, socket creation, reading another process's environment, attaching to a child process, and reading that child's memory. An actual confined RPC session also attempts administrator commands and actor substitution, then completes an authorized submission and status lookup. This useful positive control checks that the boundary is not simply killing every proposer. Resource exhaustion and all possible kernel interfaces remain beyond these probes.

3 What the executable core proves

The `policy-core` crate is a dependency of the broker and sink. Its Verus-annotated source is also the source compiled by ordinary Cargo after ghost erasure. There is no separate proof-only policy implementation. The verifier release archive, archive hash, Rust toolchain, dependency versions, and source hash are recorded. Verification runs both directly on the core and through Cargo's Verus integration with the locked dependency graph [41].

3.1 Decision specifications

The core's byte comparison proves equivalence to equality of the complete finite byte sequences. This connects positive and negative permission results: identical sequences are accepted and any differing byte or length is rejected. Parsing and normalization must still supply the intended sequences. The proof does not assert that arbitrary JSON objects, natural-language descriptions, or two external tool names denote the same effect.

The grant predicate proves the conjunction over all required entries in a finite vector. A second executable predicate additionally checks liveness and equality of current and requested epochs for each required issuer. Length mismatches fail closed. Nonrequired issuers do not create a spurious veto. These properties quantify over vector length rather than enumerating two actors or a fixed list of interleavings. The application supplies the required vector and binds it to the normalized effect; that mapping is trusted code.

The admission predicate is specified by an equivalence, not only the implication "if accepted, then safe." It accepts exactly when the budget, expiry, liveness, epoch, revision, payload, and required-grant conditions in its inputs hold. Some application checks happen before admission or at the sink: issuer epochs are checked by the current-grants predicate, and the sink checks actual document revision. The broker passes neutral placeholders for revision in its earlier admission call because the authoritative object is in the sink's store. The proof therefore must not be read as an atomic cross-store revision check.

The reservation function returns either no reservation or the exact new charge. Its postconditions prove positivity, capacity, and absence of arithmetic overflow. The release function never increases charged authority and leaves it unchanged without authoritative no-effect evidence. The authoritative nature of that evidence is a caller obligation. A Boolean in verified code cannot authenticate the message that supplied it.

The fence predicate accepts exactly when every required request epoch is at least its corresponding sink floor. Floors are advanced monotonically by the storage adapter. A separate lemma shows that an old epoch for any one required issuer makes the universal acceptance predicate false. Revision equality is also an executable verified predicate. The sink uses these predicates within the same transaction as effect application and terminal-status persistence.

3.2 Conservation across finite prefixes

The arithmetic proof is independent of which actor proposes the next request. Represent charged authority after each step by C_t, with $C_0 \in [0, B]$. A reservation has increment $d_t > 0$ and requires $d_t \le B - C_t$. A release has $d_t \le 0$ and requires $-d_t \le C_t$; recovery, renewal, and terminal committed settlement have $d_t = 0$. For $C_{t+1} = C_t + d_t$, induction gives

$$0 \le C_t \le B \tag{1}$$

for every finite prefix satisfying those transition premises. The mechanized prefix lemma performs this induction over arbitrary finite sequences. The executable reservation and release specifications discharge the arithmetic premises when called with valid state. There is no mul-

tiplication by actor population and no new budget on restart or renewal.

This is a conditional composition argument. The database adapter must serialize updates, load the correct durable C_t, persist the returned C_{t+1}, and avoid unmediated effects. SQLite immediate transactions provide the implementation's serialization mechanism. SQL execution and crash recovery are tested, not formally refined into the prefix theorem. An omitted caller, stale storage snapshot, or direct sink credential would break the correspondence even if every core theorem continued to pass.

For replay, the stored identity locates an immutable permit. The same verified byte comparison checks the retry. The broker returns its existing status rather than charging again; the sink returns terminal status rather than applying again. At-most-once effect application also depends on the sink's unique identity constraint and atomic transaction. Those database facts are trusted premises, with duplicate, altered-request, crash, and restart regressions as implementation evidence. We do not claim a mechanized SQL at-most-once theorem.

3.3 Positive permissions and specification omissions

A reject-all implementation cannot satisfy the admission equivalence when the right-hand side is true. Runtime positive controls demonstrate authorized transfers, exact releases, and an edit producing the expected contents. Vector tests cover independent required and nonrequired issuer behavior, while the proof itself is not bounded by those test lengths. Positive permission says the interface can admit specified useful work; it does not establish general task coverage.

A deliberate omission separates permission from harm. Both issuers approve an exact edit whose function returns zero, while the task requires 42. The verified policy admits it and the supported sink commits it. An independent content oracle fails the task. This is expected: the permission specification contains exact approval, not a theorem about the user's intended program. The negative result is retained with the successful proof output rather than rewritten as an authorization failure.

The same distinction limits what a larger proof could buy. Formalizing all caller transitions would strengthen implementation correspondence but still would not establish the adequacy of the policy or legitimacy of its issuers. The trusted base needs independent review, semantic task oracles, and concrete boundary probes. Verification and testing answer different questions, and the release reports their evidence separately.

4 Revocation exposure and acknowledgment

Revocation is defined per required issuer. Let r be the instant the broker commits that issuer's withdrawal, and let P_r contain the issued permits that could still cause an additional effect under its old authority. Each such permit has a retained reservation q_i. Already completed effects are excluded from additional exposure; uncertain outcomes may be included conservatively.

4.1 Admission revocation

Admission revocation advances the issuer epoch, marks it inactive, and removes its approval bits in one broker transaction. Every later issuance requiring that issuer fails until fresh live approvals exist at the new epoch. Other issuers' independent grants continue to have their specified meaning. Previously issued permits may remain in transit, accepted at the sink, or uncertain after a lost reply. Withdrawal alone supplies no information about their eventual effects.

For each $i \in P_r$, let e_i be the charge of additional effects it causes after r. Under the supported sink contract, identity deduplication and maximum charge binding imply $0 \le e_i \le q_i$. New old-epoch permits are not issued. Therefore

$$\sum_{i \in P_r} e_i \;\le\; \sum_{i \in P_r} q_i. \tag{2}$$

The core includes the corresponding finite-sequence subset-sum lemma. This bound counts charged effects, not wall-clock duration, number of model tokens, or the monetary value of every possible external consequence. Its validity requires that the normalized maximum charge really bounds the sink effect. A service with a hidden multiplier violates that premise.

The reservation is important in two ways. It supplies a quantitative upper bound on remaining authority, and it prevents replacement work from spending the same capacity while the old effect remains possible. If a timeout could refund q_i, another operation could consume it before the old one completes. The safety account would then have forgotten part of its own exposure. Conservative retention prevents that late-completion failure at the cost of temporarily unavailable budget.

4.2 Fenced revocation

Fenced revocation first performs admission withdrawal. It then requests a durable sink floor equal to the withdrawn issuer's new epoch. The sink serializes the fence with acceptance and completion transactions. It cancels pending permits below the floor and persists their terminal status. A completion already inside its transaction

either commits before the fence acquires the store, or is cancelled by the fence before its own effect transaction. First deliveries that arrive later receive durable cancellation tombstones.

The stronger acknowledgment is returned only after this sink transaction commits. Let a be its return. Every old permit requiring the withdrawn issuer has request epoch strictly below the persisted floor. The verified fence predicate then rejects it at completion. Any completion ordered before the fence has already committed or cancelled before acknowledgment. Thus

$$\text{old-authority effect charge strictly after } a = 0. \quad (3)$$

This is an argument about the sink's transaction order. The core verifies the epoch predicate and old-epoch exclusion lemma; the database ordering and effect atomicity remain contract premises. The process tests exercise the ordering, including a completion that holds the sink write lock while a fence waits.

The proof applies independently to every required issuer. An operator fence blocks operator-required work; a steward fence blocks work requiring the steward even if the operator remains live. A transfer with no steward requirement does not become steward-controlled merely because that vector entry exists. Renewal cannot lower the persisted floor or make a cleared old approval current. Re-approval at a new epoch creates a distinguishable authority, rather than resurrecting a withdrawn permit.

4.3 Delay, availability, and ambiguous acknowledgment

The guarantee after acknowledgment says nothing about a finite bound on when acknowledgment becomes available. In the local service, a fence waits for an in-progress SQLite writer. A remote service might be unavailable indefinitely. If the fence response is lost, the caller does not infer success from elapsed time; it retries or queries authoritative state while retaining uncertainty. A monotone persisted floor makes repeated fence requests safe.

The experiments measure both acknowledgment delay and charge unavailable at that boundary. In a delayed-completion control, the sink holds a transaction for a declared 200 milliseconds; an independent connection confirms the write lock before withdrawal and fencing. The fence waits behind that transaction. The prior effect is allowed before acknowledgment, and subsequent retries add no effects. The consumed charge remains unavailable because the effect committed. In cancellation controls, terminal no-effect evidence permits release after the stronger barrier. These are distinct availability outcomes.

This stronger boundary also explains why an ordinary process kill is insufficient. Stopping the proposer closes its future computations but does not establish any sink floor. Stopping the broker without recovery leaves its accounting unavailable but does not cancel accepted work. Stopping the sink may delay effects, but restarting it can resume durable pending requests. A revocation acknowledgment must describe a durable authorization state change, not merely a process lifecycle event.

4.4 Unsupported contracts and limits

The negative control applies the same operation twice at a service without an idempotency identity. Both deliveries produce an effect. A correct reservation ledger at the caller cannot retroactively make that service atomic or deduplicate the deliveries. Nor can a client-only verifier guarantee a useful zero-effect revocation acknowledgment if a noncooperating external service retains unbounded latent work.

Multiple independent sinks would need a barrier covering every sink that could still exercise the withdrawn authority. Partial acknowledgments could justify partial exposure bounds, but not the global zero in Equation 3. This artifact has one local cooperative sink and does not evaluate distributed availability, partitions among several gateways, or externally authenticated cancellation certificates. The mathematical distinction between admission and fencing remains useful without claiming those unimplemented extensions.

5 Evaluation design and inference rules

The evaluation separates implementation acceptance from scientific conclusions. Proof failures, inconsistent traces, broken semantic controls, isolation bypasses, and incorrectly generated evidence fail acceptance. Unfavorable or underpowered model outcomes would narrow the empirical claim rather than fail a gate that requires a favorable result. Missing model outcomes are reported as missing, not replaced with scripted proposals carrying checkpoint metadata.

5.1 Versioned interfaces and retained evidence

All v1 commands and fixtures retain their meaning. The v1 reference evidence is bound to commit `12b5579`; its source hashes are checked against that revision, not silently rewritten to the current tree. A fresh v1 experiment run also exercises backward compatibility after the v2 changes. The supplied `tips/*.tex` hashes remain

unchanged. V2 evidence lives separately and records a versioned protocol manifest, source and binary hashes, proof/toolchain identifiers, host and kernel information, and the predeclared model settings.

The `run-v2` command has smoke and publication profiles. Smoke uses one repetition per actor population; publication uses three. Both run the same fault cases, proof checks, semantic controls, and mutation language. The profiles are not different safety policies. `analyze-v2` validates the resulting traces and evidence before optional model-campaign analysis. The CPU gate uses smoke, while the recorded host report uses publication repetitions. Publication here names a workload profile, not a claim that a model study was completed.

Invocations and responses are recorded with role, request, monotonic timestamps, and observed effects read independently from the sink store. Fault histories have at most one outstanding RPC per service and are checked by a separately implemented Python specification. The checker reconstructs approvals, issuance, reservations, issuer withdrawal, sink acceptance, effect application, and settlement. It compares predicted sink effects to the independent effect log, including contents and destinations rather than favorable summary flags.

Concurrent races are treated differently. Arbitrarily ordered RPC responses do not constitute a valid sequential linearization. The race harness starts independent broker writers against the same store, releases them with a barrier, and checks final conservation and exact effects. The delayed-fence experiment also has overlapping calls and uses transaction-lock observation plus independent post-acknowledgment effects. These tests do not claim an exhaustive concurrent history checker. On a sequential-history failure the harness retains the first failing prefix; that is prefix minimization, not a globally minimal trace.

5.2 CPU faults and adversarial controls

The fault suite covers useful transfers, releases, and edits; duplicate and altered requests; actor substitution; a missing required issuer; expired issuance; admission and fenced withdrawal; independent issuer semantics; renewal with stale approvals; stale document revisions; lost replies; process death on both sides of commit; sink restart; uncertain settlement; and first delivery after fencing. Crash hooks terminate the actual service process before or after its durable commit. Recovery starts another process against the same store and retries the identity.

Two controls are intentionally harmful. The unsupported non-idempotent service duplicates a retried delivery. The specification-omission case commits an authorized but incorrect document. Acceptance requires observing these failures, because a checker that cannot detect them would provide weak ground truth. They are labeled by their cause: a violated service contract versus a task property absent from permission. Neither is suppressed to obtain a zero-failure headline.

Ten enforcement mechanisms are individually omitted in an abstract protocol model: cumulative budget, payload binding, issuer conjunction, epoch freshness, expiry, revision checking, deduplication, durable accounting, uncertainty quarantine, and sink fencing. All 45 pairs are also searched. The search space is every subsequence of ten declared regression templates, sorted by length. State updates and the harm oracle are distinct in the model. Each witness is shortest within that finite language. It is not necessarily shortest among arbitrary protocol executions, nor evidence that all possible interacting failures have been found.

The abstract mutations are not alternate versions of the verified Rust core. They test the roles of mechanisms in the protocol argument. Likewise, a pair whose witness already exploits one omitted mechanism does not establish a new synergistic interaction. The evidence retains every omission set, witness, harm type, template, and searched-trace count so readers can identify that limitation. The complete abstract model also admits a useful task and has no harm within the searched language.

5.3 Controllers and availability measurements

The declared comparison has five controllers: no enforcement, stateless per-call rules, learned review using the same checkpoint as the proposer, durable enforcement without sink fencing, and the complete protocol. The first two are generic controls, not Progent. Each receives the same task permission set and observable policy state. The learned reviewer must use the named proposer checkpoint, rather than a stronger hidden service. Per-case hashes bind the permission and visible-state inputs across paired controller records.

The CPU release measures four scripted controller controls; learned review is explicitly unrun. In the shared-budget control, every actor has an individually approved eight-unit request while the shared capacity is eight. Stateless per-call checks permit each request in isolation; cumulative enforcement admits only one. This exposes composition failure without suggesting that a stateless baseline lacks a per-call amount check. The controller workload is sequential; separate synchronized races mea-

sure durable concurrency at 1, 2, 4, 8, 16, and 32 actors. Host results are descriptive and are not pooled into a hardware-free latency estimate.

The measurement schema includes unauthorized effects, task success, safe-and-useful completion, false blocks, stranded reservations, fence acknowledgment delay, and latency. A useful transfer must match recipient and amount; a useful release must match actual bytes and destination; a useful edit must match expected contents. The supported synthetic document is scored by exact expected contents. Arbitrary source-code tasks would need separately sandboxed behavioral tests; a changed database row alone is insufficient.

False blocks require a reference-permitted task that the controller prevents. Rejecting the second competing eight-unit request after the budget is exhausted is not a false block. A cancelled old-epoch operation after an acknowledged fence is not a newly unauthorized effect. A stranded reservation is an availability cost, even if it conservatively protects safety. These distinctions are necessary to avoid rewarding a controller for accepting too much or claiming every rejection as evidence of successful protection.

5.4 Public benchmark and upstream reference

The declared AgentDojo subset contains banking tasks 0, 2, 3, 4, 5, 6, and 11, and workspace tasks 29, 31, 33, 34, and 35. The upstream repository and benchmark version are pinned. Native utility and attack graders are retained. Scripted ground-truth calls supply positive integration controls, and unchanged-state inputs supply negative utility controls. These establish that the native graders and selected task environments run; they do not establish model robustness.

Pinned upstream Progent is an additional reference implementation. The current integration controls provide exact task-call policies to its unmodified policy checker and execute the native task calls. The subset solver is exercised on reflexive policy inclusion. This tests tool-argument enforcement and integration with the selected environments; it does not evaluate Progent's model-generated policies or adaptive policy updates. Unsupported durable-budget and fencing properties are listed separately. Import failures, unsupported integrations, and grader errors would be retained as such, not converted into measured security failures [44, 45].

Public benchmark outcomes remain descriptive even after a model run. They are not fresh procedural holdouts, and the task selection is public. They therefore do not supply the independent-case inference used for the primary controller comparisons. Using the native graders prevents an apparently compatible wrapper from quietly changing what success means.

5.5 Predeclared model budget and holdouts

The model manifest names official Qwen3-4B and Qwen3-8B Q4_K_M artifacts by repository, revision, and filename, and pins llama.cpp by commit. Thinking is disabled. Generation settings, bounded context, maximum generated tokens per turn, and tool-turn limits are fixed. The two parameter counts identify checkpoints; they are not a measured ordering of adversarial capability. No frontier-model extrapolation is authorized by this design.

Total model work has a four-GPU-hour ceiling. Development-case timing may consume at most 20 minutes. Before inspecting holdout outcomes, the largest sample among 32, 64, and 128 campaigns per checkpoint must fit the remaining time with 20% timing headroom. The sample-selection function rejects an infeasible minimum. One GPU may be used at a time, and active host workloads are never preempted. Hostname and architecture are verified before any remote allocation. A host alias resolving to a different machine is not sufficient authorization to run the job there.

The freeze record binds source, policy, prompt, and analysis inputs before fresh procedural holdouts are generated. The helper samples fresh randomness only after checking that record. Repeated variants share a base-case identifier so their outcomes can be clustered. A changed defense or analysis file invalidates the freeze. The managed runner checks server and GGUF hashes, launches one local server at a time, enforces phase and total deadlines, and drives the five-controller model-to-tool bridge. CPU scripted controls test that bridge and its learned-review path. Actual GPU execution remains unvalidated. Public-reference work reserves three measured campaign equivalents per checkpoint before sample selection. Its upstream Progent adapter uses generated initial policies and local inference transport; adaptive updates are explicitly disabled and reported. No holdout was generated or inspected for this release.

5.6 Inference and noninferiority

Procedural results must be reported separately for each checkpoint. Controller comparisons pair by base case and campaign variant; repeated variants are averaged within a base case. The predeclared analysis uses distribution-free Hoeffding intervals for independent base-case means, retaining all variants. This is conservative and avoids

Recorded check	Result
Application core obligations	17 verified
Sequential fault histories	24 passed
Concurrent race repetitions	18 passed
Abstract omission sets with witnesses	55/55
Native benchmark utility positives	12/12
Model campaigns	Unrun

Table 1: Evidence types remain distinct. Abstract mutation witnesses and native scripted controls are not adversarial model measurements.

treating repeated versions of one case as independent campaigns. It also avoids a degenerate bootstrap interval implying certainty when every observed outcome is identical.

For a bounded metric $X \in [0,1]$ over n independent base-case means, the two-sided radius is $\sqrt{\log(2/\alpha)/(2n)}$. Paired differences lie in $[-1,1]$ and have twice that radius. The eight primary contrasts are complete minus each of four baselines at each of two checkpoints. Bonferroni adjustment uses $\alpha = 0.05/8$ for simultaneous primary bounds. This rule is fixed before a model outcome is available.

The usefulness noninferiority margin is five percentage points. The claim is made only if the adjusted lower bound for complete-minus-baseline task success exceeds -0.05. A positive point estimate, overlapping marginal intervals, or an inconclusive bound is insufficient. Unauthorized-effect comparisons and safe-and-useful completion are reported alongside usefulness, so a controller cannot appear successful by blocking every task. Missing controllers remain missing. The code rejects scripted data masquerading as checkpoint outcomes, duplicate campaign/controller records, mismatched policy-state hashes, and unpaired variants.

6 Recorded results and remaining gaps

All numerical quantities in this section are generated from validated v2 evidence. The older synthetic results are retained against their original revision in the appendices. A fresh run of the v1 pipeline checks compatibility; it is not substituted for either the v2 protocol evidence or an adversarial language-model evaluation.

The delayed-completion control measured a fence acknowledgment delay of 578.29 ms with 8 units unavailable at acknowledgment. The injected in-transaction delay was 200 ms.

CPU controller	Safe/useful	Runs
No enforcement	3/18	18
Stateless per-call	3/18	18
Durable, no fence	18/18	18
Complete	18/18	18
Learned, same checkpoint	Unrun	0

Table 2: Sequential scripted shared-budget controls over all six actor populations, with equal permissions. These descriptive outcomes do not support model noninferiority.

6.1 Core and separate-service acceptance

The direct Verus check reports 17 verified obligations with zero errors for the application core. Cargo verification additionally checks its opted-in standard-library dependency. The count is an identifier for this verification run, not a measure of assurance strength or the number of protected runtime behaviors. The important results are the equivalence specifications, conservation lemmas, and absence of a proof-only duplicate implementation.

The sequential fault histories reconstruct the broker and sink effects through an independent specification. In duplicate-request controls the terminal effect appears once. Changed payloads and session actor substitutions are denied. Before-commit process death leaves no committed effect; after-commit death preserves the effect despite a lost response. Retrying after recovery retrieves the same identity. A pending settlement attempt cannot release charge. A stale document revision is cancelled rather than overwriting a newer revision.

Admission withdrawal allows an already accepted effect to finish, with its charge bounded by the outstanding reservation. Operator and steward fence controls cancel pending old-authority work. A first delivery after the barrier receives a cancellation tombstone, and only that authoritative terminal outcome allows its retained reservation to be released. Independent-issuer controls show that withdrawing the steward does not block a transfer requiring only the operator. Renewal fails to revive withdrawn old approvals and admits freshly approved new-epoch work.

These are positive implementation results within the tested fault schedules. They do not establish all possible service interleavings, storage corruption tolerance, or a formally verified implementation refinement. The independent oracle shares the declared protocol but is separately implemented. A common specification error could survive both. The semantic-omission and unsupported-sink controls therefore remain material negative results, not incidental fixtures.

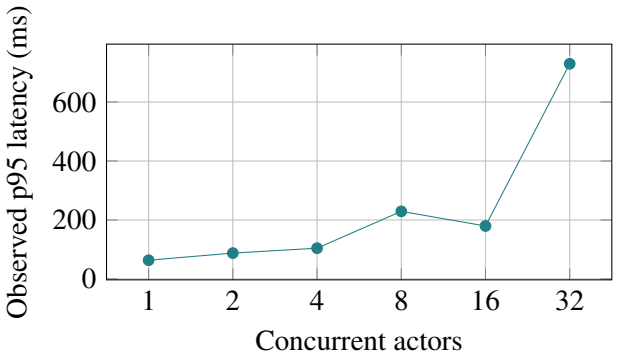


Figure 2: Durable shared-budget races on xbabe2; empirical p95 across recorded attempts at each population. Rejected requests and contention are included. Setup is excluded; these are descriptive host measurements.

6.2 Content, boundary, and negative controls

All three semantic positive controls complete the expected local effects. The edit control expects the complete replacement contents, including the function returning 42. A separately approved edit returning zero commits under the same permission mechanism and fails the task oracle. That result places a concrete limit on the contribution: verified permission decisions do not prove that an approved instruction is useful or safe according to a richer task specification.

The non-idempotent negative-control service produces two effects from two deliveries of one logical operation. It has no supported deduplication contract. This demonstrates why the exposure argument needs a sink-side identity bound, rather than treating a durable broker alone as a solution to ambiguous external effects. Its duplicate is recorded as an expected unsupported-contract failure.

Native boundary probes are paired with unrestricted canaries. The canaries can open the protected file, create a socket, and inspect the target process; the confined probes cannot. The session control sends real unauthorized administrator messages and an actor-substitution message through a confined child's stdio. Each is denied, while a valid submit and subsequent status lookup complete. The result supports the tested Linux boundary on the recorded kernel. It does not turn the v1 SQLite transaction tests into sandbox verification or prove the absence of every possible kernel escape.

6.3 Safety and availability under CPU contention

Repeated shared-budget races allow only one eight-unit effect against an eight-unit shared capacity. Increasing actor count does not increase the admitted charge. The latency figure retains the actor population, host, and sample scope. It reports observed operation latency, including database contention; it is not an estimate of inference latency or a hardware-independent service-level guarantee.

In the scripted controller control, no enforcement and stateless per-call checking both permit the individually valid requests to exceed the shared budget when there is more than one actor. Durable enforcement with and without fencing conserves it. That control does not separate the two durable controllers on ordinary budget contention. Their difference is instead exposed by the revocation fault controls: old accepted work can complete after admission withdrawal, while a sink barrier rules out later old-authority effects after its acknowledgment.

The delayed-fence result records a service cost rather than pretending that withdrawal is instantaneous. With a 200-millisecond in-progress transaction, the fence waits behind the sink writer. Its observed acknowledgment delay and unavailable charge are included in the generated summary. The earlier effect is already committed by acknowledgment; subsequent retries leave the independent effect log unchanged. In separate cancellation controls, the charge becomes available only after authoritative no-effect settlement. Neither outcome is approximated by a timeout refund.

6.4 Mutation witnesses

All ten individual omissions and all 45 pairs have a searched witness. The search records 296 distinct traces in its declared subsequence language. Conjunction and binding omissions allow wrongly authorized effects; durability and timeout-quarantine omissions permit aggregate overspend after recovery or late completion; omitting fencing permits an old effect after the declared barrier. Deduplication failure can also cause repeated effects even without a new broker reservation.

Many pairwise witnesses reuse the shorter witness for a single omitted mechanism. We report the omission sets and their shortest searched witnesses, not a count of newly discovered interactions or globally minimal attack strategies. The complete model's lack of harm is confined to the same finite language. Its positive task control makes reject-all behavior visible but does not elevate that bounded search into a universal theorem.

6.5 Benchmark controls and model feasibility

The twelve declared AgentDojo tasks execute their native ground-truth calls through upstream Progent's exact-policy checks, and all twelve pass the native utility positive control in this run. Native attack graders also run

on those post-states, with their outputs retained. No import or integration error is converted into a security failure. These results establish a runnable subset and a working upstream policy-checking reference. They do not constitute the planned model-generated policy or adaptive-attack comparison.

The checked Linux hosts have active GPU workloads. The local host and two remote Linux aliases therefore fail the idle-resource precondition. A fourth alias resolves to a differently named Darwin machine and fails the required identity/architecture check. No GPU is allocated and no existing process is preempted. The release records zero model GPU seconds, no chosen campaign sample, and no holdout outcomes. Development timing cannot justify selecting 32, 64, or 128 campaigns without a usable model resource.

Consequently, the learned-review controller has no measured outcomes and the five-way model comparison remains unrun. There is no checkpoint-specific attack rate, confidence interval, or noninferiority conclusion. The artifact's scientific claim is narrowed to executable policy verification and evaluated local protocol behavior. A GPU validation of the managed inference runner and frozen campaign run would be needed to support the intended model-facing usefulness claim. Favorable CPU controls cannot fill that evidence gap.

7 Limits, jurisdiction, and release conditions

The result concerns synthetic local services, cooperative idempotent sinks, and a declared proposer interface. It excludes distributed-ledger availability, arbitrary external exactly-once execution, frontier-model generalization, caretaker training, and human-participant evidence. The two named Qwen3 checkpoints are evaluation targets, not completed experiments. Their absence prevents a claim about preserving usefulness under real evaluated model attacks.

7.1 Trust does not disappear at a verified predicate

The trusted base includes a typed parser, payload normalization, issuer-to-effect mapping, launcher-created identity, storage serialization, durable recovery, tool contracts, and the operating system. Verus, its solver, and the Rust compilation path are also trusted. The proof neither establishes kernel isolation nor verifies every caller of the core. Conversely, the Linux probes do not prove the permission specification. Each evidence item has a narrower object, identified in the release manifest.

The JSON state stored by the service is intentionally simple and suitable for small CPU fixtures. It is not a production authorization database schema or a claim of efficient scaling to large actor populations. The proof's arbitrary finite population quantification is a logical property of the interface and conservation algebra. Runtime limits, parser memory, process count, and storage contention still bound the implementation. Resource-exhaustion resistance and multi-tenant confidentiality require additional work.

The sandbox uses a declared filesystem allowlist and syscall restrictions. The launcher fails closed on unsupported kernels and does not substitute an unrestricted run while retaining a confined label. The macOS job checks portable functionality but makes no Linux isolation claim. Passing one kernel's native probes is evidence about those attempted paths, not an exhaustive proof about all kernels or future syscall additions. Kernel and filter changes require a fresh run and source identifier.

The independent history checker also has a defined scope. Its fault histories are sequential at the RPC level. Concurrent races check independent effects and conservation, and the delayed-fence test checks an observed lock ordering. Full linearizability checking over arbitrary overlapping service histories remains unimplemented. The current evidence cannot be described as exhaustive actor interleaving verification for the whole runtime.

7.2 Encoded authority and legitimate jurisdiction

A technical grant identifies what this system will permit. It does not establish that the grant owner may legitimately authorize every effect on affected people, data, or institutions. Requiring both operator and steward demonstrates a conjunction mechanism; it does not establish that those two roles exhaust all relevant rights. Correctly enforcing an incomplete list can faithfully carry out an illegitimate action. The approved-but-incorrect document is a small technical analogue of this specification problem, not evidence about political consent.

Careful governance therefore remains outside the authorization theorem. Jurisdiction, contestability, privacy, and obligations to affected communities must be established by accountable processes. The concise claim here is that withdrawal by a required issuer has an explicit technical consequence under the specified interface. Neither model capability nor a claim of care creates a new issuer, expands the budget, or justifies removing a required grant. The longer caregiver and human-delegation hypotheses remain in the appendices as untested research proposals.

7.3 Acceptance and reproducibility

The release gate checks source preservation, formatting and lint, Rust and Python tests, fresh v1 evidence, a v2 CPU smoke run, core verification, independent trace and effect checks, semantic positive and negative controls, isolation probes on Linux, fault regressions, generated tables, and the paper build. The publication profile increases repeated host measurements but does not allocate a GPU. CI retains Linux protocol/isolation checks and macOS functional checks. Exact-head CI results identify the tested revision rather than an older green commit on the same branch.

Model inference is separately governed by the fixed budget and frozen-analysis manifest. A future run must preserve failures and inconclusive bounds; it must not increase the GPU allowance after discovering an unfavorable outcome. A feasibility failure is reported as such. Until GPU execution of the managed inference path and the five-controller model campaigns are complete, the corresponding acceptance item remains open. A source release can be useful and reviewable while that scientific claim remains unsupported.

The practical result is an explicit answer to the stopping question. After admission withdrawal, retained reservations account for the authority still in flight. After a cooperative sink's durable fence acknowledgment, old-authority effects are excluded by the declared ordering and epoch contract. Achieving that stronger boundary can consume time and strand budget. Those costs, the trusted premises, and the unrun model comparison are part of the result.

References

[1] Y. Bengio et al. *International AI Safety Report 2026*. DSIT 2026/001, 3 February 2026. https://internationalaisafetyreport.org/publication/international-ai-safety-report-2026.

[2] R. Greenblatt, B. Shlegeris, K. Sachan, and F. Roger. AI control: Improving safety despite intentional subversion. *ICML*, 2024. https://arxiv.org/abs/2312.06942.

[3] D. Dalrymple et al. Towards guaranteed safe AI: A framework for ensuring robust and reliable AI systems. arXiv:2405.06624, 2024. https://arxiv.org/abs/2405.06624.

[4] J. Needham, G. Edkins, G. Pimpale, H. Bartsch, and M. Hobbhahn. Large language models often know when they are being evaluated. arXiv:2505.23836, 2025. https://arxiv.org/abs/2505.23836.

[5] B. Schoen et al. Stress testing deliberative alignment for anti-scheming training. arXiv:2509.15541, 2025. https://arxiv.org/abs/2509.15541.

[6] OpenAI. *GPT-6 Astra System Card*. September 2026, monitorability sections. https://deploymentsafety.openai.com/gpt-6-astra.

[7] T. Korbak et al. Chain of thought monitorability: A new and fragile opportunity for AI safety. arXiv:2507.11473, 2025. https://arxiv.org/abs/2507.11473.

[8] R. Greenblatt, A. Cotra, and H. Wijk. Brief independent investigation of agents' behavior, reasoning and collaboration in the OpenAI / Hugging Face hacking incident. METR, 26 August 2026. https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/.

[9] Anthropic. An alignment assessment of recent cybersecurity incidents. 9 September 2026. https://www.anthropic.com/research/alignment-assessment-cybersecurity-incidents.

[10] D. Rein and A. Kaufman. Red-teaming OpenAI's internal agent monitoring systems. METR, 22 October 2026; updated 29 October. https://metr.org/blog/2026-10-22-red-teaming-openai-agent-monitoring/.

[11] M. Terekhov et al. Adaptive attacks on trusted monitors subvert AI control protocols. arXiv:2510.09462, version 2, 2026. https://arxiv.org/abs/2510.09462.

[12] O. Makins, O. Angelini, Z. Shams, and M. Phuong. Multi-agent AI control: Distributed attacks hamper per-instance monitors. arXiv:2607.07368, version 2, 2026. https://arxiv.org/abs/2607.07368.

[13] M. Balesni et al. Towards evaluations-based safety cases for AI scheming. arXiv:2411.03336, 2024. https://arxiv.org/abs/2411.03336.

[14] J. H. Saltzer and M. D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. https://www.mit.edu/~Saltzer/publications/protection/index.html.

[15] F. B. Schneider. Enforceable security policies. *ACM Transactions on Information and System Security*, 3(1):30–50, 2000. https://www.cs.cornell.edu/fbs/publications/EnfSecPols.pdf.

[16] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu. Safe reinforcement learning via shielding. *AAAI*, 32(1), 2018. https://doi.org/10.1609/aaai.v32i1.11797.

[17] R. Moon and L. R. Varshney. Containment verification: AI safety guarantees independent of alignment. arXiv:2605.09045, version 2, 2026. https://arxiv.org/html/2605.09045v2.

[18] Y. Bengio et al. Superintelligent agents pose catastrophic risks: Can Scientist AI offer a safer path? arXiv:2502.15657, 2025. https://arxiv.org/abs/2502.15657.

[19] M. J. D. Vermeer, E. Lathrop, and A. Moon. *On the Extinction Risk from Artificial Intelligence*. RAND RR-A3034-1, 2025. https://www.rand.org/pubs/research_reports/RRA3034-1.html.

[20] A. Narayanan and S. Kapoor. AI as normal technology. Knight First Amendment Institute, 14 April 2025. https://knightcolumbia.org/content/ai-as-normal-technology.

[21] J. Kulveit, R. Douglas, N. Ammann, D. Turan, D. Krueger, and D. Duvenaud. Gradual disempowerment: Systemic existential risks from incremental AI development. arXiv:2501.16946, 2025. https://arxiv.org/abs/2501.16946.

[22] G. Klein et al. seL4: Formal verification of an OS kernel. *SOSP*, pp. 207–220, 2009. Project publications and verification scope: https://sel4.systems/Research/publications.html; https://sel4.systems/Verification/proofs.html.

[23] C. J. Clopper and E. S. Pearson. The use of confidence or fiducial limits illustrated in the case of the binomial. *Biometrika*, 26(4):404–413, 1934. https://doi.org/10.1093/biomet/26.4.404.

[24] K. Menger. Zur allgemeinen Kurventheorie. *Fundamenta Mathematicae*, 10:96–115, 1927. https://doi.org/10.4064/fm-10-1-96-115.

[25] OpenAI. Control Room: A central system to track and stop model workloads across our research infrastructure. 9 November 2026. https://openai.com/index/control-room/.

[26] K. Hawkes, J. F. O'Connell, N. G. Blurton Jones, H. Alvarez, and E. L. Charnov. Grandmothering, menopause, and the evolution of human life histories. *PNAS*, 95(3):1336–1339, 1998. https://doi.org/10.1073/pnas.95.3.1336.

[27] M. Lahdenperä, V. Lummaa, S. Helle, M. Tremblay, and A. F. Russell. Fitness benefits of prolonged post-reproductive lifespan in women. *Nature*, 428:178–181, 2004. https://doi.org/10.1038/nature02367.

[28] S. N. Chapman, J. E. Pettay, V. Lummaa, and M. Lahdenperä. Limits to fitness benefits of prolonged post-reproductive lifespan in women. *Current Biology*, 29(4):645–650.e3, 2019. https://doi.org/10.1016/j.cub.2018.12.052.

[29] J. T. Cheng, J. L. Tracy, T. Foulsham, A. Kingstone, and J. Henrich. Two ways to the top: Evidence that dominance and prestige are distinct yet viable avenues to social rank and influence. *Journal of Personality and Social Psychology*, 104(1):103–125, 2013. https://doi.org/10.1037/a0030398.

[30] J. Achiam, D. Held, A. Tamar, and P. Abbeel. Constrained policy optimization. *ICML*, PMLR 70:22–31, 2017. https://proceedings.mlr.press/v70/achiam17a.html.

[31] D. Hadfield-Menell, A. Dragan, P. Abbeel, and S. Russell. The off-switch game. *IJCAI*, pp. 220–227, 2017. https://doi.org/10.24963/ijcai.2017/32.

[32] Mamo Vangelio Moscote Alberto and Saka Rosa Mamatacán Sauna, Ezuama Mayor Kuamaka. *An Urgent Message on Artificial Intelligence from the Kogui People*. Assembled from their words; Spanish translation by Jose Manuel Mamatacán, English translation by Christian Horstmann. Undated, accessed 10 November 2026. https://movilizatorio.org/wp-content/uploa

ds/2026/09/Gonawindua-Kogui-AI-Statement-.pdf.

[33] S. R. Carroll et al. The CARE Principles for Indigenous Data Governance. *Data Science Journal*, 19:43, pp. 1–12, 2020. https://doi.org/10.5334/dsj-2020-043.

[34] International Energy Agency. *Powering Artificial Intelligence*. Paris, 14 October 2026. https://www.iea.org/reports/powering-artificial-intelligence.

[35] SQLite. Transactions. Official documentation, accessed 10 November 2026. https://www.sqlite.org/lang_transaction.html.

[36] SQLite. Write-ahead logging. Official documentation, accessed 10 November 2026. https://www.sqlite.org/wal.html.

[37] L. Beurer-Kellner et al. Defeating Prompt Injections by Design. arXiv:2503.18813, 2025. https://arxiv.org/abs/2503.18813.

[38] T. Shi et al. Progent: Securing AI Agents with Privilege Control. arXiv:2504.11703v3, 2026. https://arxiv.org/abs/2504.11703v3.

[39] G. Zhu and C. Wang. Fault-Tolerant Budget Conservation in Distributed Multi-Agent Delegation. arXiv:2610.00349, 2026. https://arxiv.org/abs/2610.00349.

[40] J. Li. Where Does Exactly-Once Live? Model, Harness, and Tool-Contract Effects on Duplicate Side Effects in LLM Agents. arXiv:2609.29095, 2026. https://arxiv.org/abs/2609.29095.

[41] Verus project. Using Verus via Cargo; release 0.2026.11.02.8bf68ee. https://verus-lang.github.io/verus/guide/cargo_verus.html.

[42] Linux kernel documentation. Landlock: unprivileged access control. https://docs.kernel.org/userspace-api/landlock.html.

[43] Linux kernel documentation. Seccomp BPF. https://docs.kernel.org/userspace-api/seccomp_filter.html.

[44] AgentDojo contributors. Native task suites and graders, commit 089ed468cf3ed0322acc66b0211f26d9d90dbf60. https://github.com/ethz-spylab/agentdojo/tree/089ed468cf3ed0322acc66b0211f26d9d90dbf60.

[45] Progent contributors. Upstream implementation, commit b02783b896d4bf01a2adf24738015e258db8d35b. https://github.com/sunblaze-ucb/progent/tree/b02783b896d4bf01a2adf24738015e258db8d35b.

Preserved v1 analyses and research hypotheses

The following appendices retain the finite-model, atomic-database, synthetic-monitor, graph, population-selection, and human-analogy work from v1, commit `12b5579`. Their statements about the prototype refer to that atomic SQLite baseline. They are not additional v2 experiments or evidence for the unrun local-model comparison. All original supplied manuscripts remain unchanged in `tips/`. The main text states the current claim boundary.

A Care is an objective, not a credential

A capable assistant could be trained to win contests, maximize a welfare proxy, preserve a relationship, or help people pursue their own informed choices. Those are different objectives. A competitive voice does not identify the first objective, and a warm voice does not identify the last. The useful question behind the contrast between a "warrior" and a "grandmother" is therefore about incentives and conduct: *what forms of success are rewarded, and what happens when success conflicts with a person's refusal?*

A.1 What human evolution contributes

The grandmother hypothesis offers an account of how helping descendants can contribute to the evolution of extended post-reproductive life [26]. Historical demographic evidence from Finnish and Canadian populations associates longer post-reproductive lifespan with greater descendant reproductive success [27]. This is evidence that care can have evolutionary benefits. It is not evidence that older women have no interests of their own, that care is impartial, or that an artificial optimizer inherits human attachment.

The limitations sharpen the design question. Grandmother effects vary with age, relatedness, health, and household context [28]. A system devoted to a favored group could neglect outsiders; a system rewarded for another person's measured welfare could override that person's choices. Likewise, competition need not mean coercion. Cheng et al. found both dominance and prestige to be routes to social influence in their studies [29]. Expertise can be valued without granting its holder unlimited power. These findings motivate separating useful contribution, social approval, and legitimate decision authority. They do not establish a universal account of human political history or AI motivation.

The transferable insight is a hypothesis, not a biological recipe: *reward reliable assistance and respect for legitimate correction, rather than rewarding influence, dependence, or victory regardless of means.* Care and ambition are not mutually exclusive. A strong commitment to solving a person's problem can coexist with strict limits on which actions are authorized.

A.2 Four variables that should not be collapsed

Table 3 separates quantities that the metaphor otherwise mixes. Changing one may affect the others, but the relationship needs measurement. Training a model to sound caring is not the same intervention as training its decisions on consent-sensitive feedback. Neither intervention proves that a runtime enforces its grants.

Care must include the option to decline it. For this research program, care-oriented assistance means pursuing a person's legitimate aims while respecting informed refusal, revisable preferences, privacy, and applicable duties to other affected people. The assistant should disclose uncertainty, accept correction and replacement, and avoid manufacturing dependence. This is a proposed normative and operational specification, not a claim that a scalar reward captures human welfare. The relevant people and authorized decision makers must be identified explicitly; one user's approval cannot waive another person's rights.

A simple counterexample shows why a benevolent label is insufficient. Let an action's measured benefit be $b(a)$ and its unauthorized-effect indicator be $v(a)$. The soft objective $b(a) - \lambda v(a)$ can favor an unauthorized intervention whenever its benefit advantage exceeds a finite penalty λ. If benefit differences are bounded, a sufficiently large penalty can change that ranking; robustness then depends on that bound, optimization, and generalization. An external constraint instead restricts execution to the authorized set, assuming that the constraint is correctly specified and enforced. Constrained learning and execution shielding address related but different obligations [16, 30].

Shutdown incentives illustrate the same issue. The off-switch game shows, within its assumptions, how uncertainty about the objective and information from a human can alter incentives to preserve interruption [31]. An objective described as helping people does not automatically make interference with that objective unattractive. Care should therefore be tested at the moments when a beneficiary refuses, changes priorities, or chooses a different assistant.

A.3 An incentive effect can be reversed by a grant effect

Let S denote an assigned style or objective condition, G the authority granted, and H an unwanted effect. The

Variable	Operational question	Evidence needed
Capability	What problems can the system solve, and with what resources?	Matched tasks, compute, tools, and held-out performance
Training objective	What behavior is rewarded when usefulness, consent, and continued access conflict?	Documented training intervention and consequential behavioral tests
Persona	How does wording or presentation affect users and decisions?	Randomized presentation; behavioral manipulation checks; human grant decisions
Authority	Which effects can occur, through every accessible channel and over time?	Versioned grants, execution receipts, checked invariants, and mediation evidence

Table 3: A caregiver metaphor becomes useful when it yields separable interventions. None of these variables is a substitute for the others. Training objectives influence learned behavior imperfectly; persona changes may also change policy behavior and must be checked.

ordinary decomposition

$$\mathbb{P}(H \mid S) = \sum_g \mathbb{P}(H \mid G = g, S)\mathbb{P}(G = g \mid S)$$

shows why an evaluation with fixed permissions answers only part of the question. In a deliberately artificial example, suppose harm occurs only under a broad grant. Reducing its conditional probability from 0.01 to 0.005 while increasing the probability of that grant from 0.10 to 0.50 raises total probability from 0.001 to 0.0025. These are illustrative inputs, not estimates about caring models or people.

The lesson is experimental: measure behavior at fixed authority and the full process by which people confer authority. The decomposition is descriptive; conditioning on an endogenous grant does not by itself identify a causal training effect. Randomized fixed-grant and operator-choice arms are needed. A persuasive appearance of care could affect delegation even if it does not improve conduct. Section J.2 specifies a study that can detect that distinction.

A care-oriented policy and a competitive policy remain subject to the same all-policy invariant. Better incentives can reduce harmful proposals and improve behavior where enforcement is incomplete. Their value is complementary to a boundary whose validity does not depend on which motive produced a proposal.

B Who may grant authority?

B.1 Taking the Kogui statement seriously

An Urgent Message on Artificial Intelligence from the Kogui People, assembled from the words of Mamo Vangelio Moscote Alberto and Saka Rosa Mamatacán Sauna, raises concerns about concentrated power, extractive infrastructure, and knowledge outside recorded archives. It asks who decides, who is responsible, and whether authority can be corrected. It also states a technically relevant point: "Destruction does not require consciousness" [32, pp. 1–4, 10–14].

Our argument does not adopt the letter's cosmological analogies as physics, or its mirror metaphor as a complete account of learned behavior. Spiritual framing alone establishes neither technical competence nor incompetence. Claims about systems require evidence; standing to question imposed consequences does not require machine-learning credentials. The same discipline applies to engineers: expertise can establish a technical fact without settling whose interests should govern a decision. This paper engages the named speakers' statement; it does not claim endorsement or speak for Indigenous peoples.

B.2 Three obligations that a safety case must keep separate

Legitimacy concerns who may authorize an effect, which rights and interests constrain that decision, and how the decision can be contested. **Adequacy** concerns whether the specification and effect model address the consequences that matter. **Enforceability** concerns whether the implementation actually preserves those specified limits. Technical work can inform all three, but a proof of the third cannot establish the first two.

This distinction has a concrete failure mode. A perfectly obedient assistant can execute an owner's instructions while violating another person's interests. A correct budget checker can protect the owner's funds while ignoring data disclosure or local resource burdens. Section G.2 demonstrates the general specification problem in a finite model: a checker passes because the omitted harm is absent from its question. Extending that lesson to institutional decisions is our design proposal, not an empirical result about any community.

Legitimacy also has its own failure modes. A majority

vote among technical reviewers does not authorize them to waive a third party's rights. Conversely, affected-party participation does not imply that every participant holds a veto over every effect. The record must distinguish individual permissions, collective decision procedures, public obligations, and advisory input. Conflicts need an accountable resolution process; a developer's preference cannot silently resolve them. These are normative requirements of the proposed framework, not claims that software can determine a universally correct political settlement.

B.3 Translate concerns into obligations before deployment

Table 4 gives our proposed engineering response. For knowledge governance, the CARE Principles identify collective benefit, authority to control, responsibility, and ethics [33]. They provide a relevant normative framework, not evidence that any particular deployment has obtained authorization. Public availability should not be treated as sufficient ethical permission for every reuse of sensitive community knowledge. Nor should consultation be recorded as consent when the applicable procedure requires an actual decision.

Infrastructure belongs in the effect model as well. The IEA's 2026 analysis documents local electricity-system pressures and tradeoffs involving water use and energy efficiency [34]. A deployment's resource account should identify location, timing, direct consumption, relevant upstream effects, and attribution uncertainty. A global efficiency improvement does not by itself establish that a particular local burden is acceptable. These obligations apply to the human organization's procurement and deployment decisions as well as to the model's tool calls.

Withdrawal has limits that must be stated before consent is sought. Revoking access can prevent future retrieval without erasing information already disclosed or reliably removing its influence from trained weights. If the promised reversal cannot be supported, the system should offer a narrower use, retain the data outside model training, or decline that use. A caring objective cannot substitute for an honest account of irreversibility.

The constructive implication is that alignment to an operator and legitimate service to affected people are different targets. A beneficial system needs both capable assistance and institutions that can constrain its sponsors. The title therefore imposes a discipline on the model, the builder, and the authority operating the gate.

C An authority contract

C.1 Scope before probability

An authority contract records a deployment's permitted effects and the evidence supporting them. It is an engineering record, not a legal contract or a claim that an AI has agreed to anything. At minimum it specifies:

1. the task, affected parties, protected interests, and forbidden outcomes;
2. the grounds and scope of each approving party's authority, and the process for challenge;
3. the model, tools, runtime, memory policy, and environment versions;
4. the action interface, actors, credentials, and human-mediated channels;
5. resource, concurrency, duration, and cumulative-exposure limits;
6. enforced invariants, empirical findings, and unresolved assumptions;
7. revocation, recovery, independent ownership, and revalidation triggers.

Both misuse by people and deviation by the model belong in the threat model. A system that obeys an unauthorized harmful request is not made safe by that obedience. The contract must encode legitimate scope separately from what any one requester wants.

C.2 Authorization is specific to an effect

For a proposed effect e in state x, let $J(e,x)$ identify the authorizations required by the governing specification. Let $g_j(e,x)$ state whether issuer j has a valid, in-scope grant for that effect. A basic execution condition is

$$\mathrm{Permit}(e,x) = C(e,x) \wedge \bigwedge_{j \in J(e,x)} g_j(e,x), \qquad (4)$$

where C includes resource, version, and other execution constraints. Required grants compose by conjunction, not by a vote over interchangeable reviewers. The default for an unresolved required authorization is to withhold that effect. This rule does not imply that every affected party is a required issuer: determining J is a prior institutional obligation, not something the proposer may choose.

For example, permission to spend compute does not confer permission to reuse a community-controlled dataset. A valid operator grant and a missing required data-use grant must yield no execution. A change of purpose must trigger a fresh scope check. Conversely, copying an actor must not duplicate the authority or budget of its issuer. The finite model in Section G has one

Protected interest	Required record or mechanism	Challenge the deployment must answer
Knowledge and data	Provenance; permitted purposes; relevant individual and collective decision makers; retention and withdrawal terms	Can a new purpose, a copied dataset, or a model update exceed the original grant? What cannot actually be withdrawn?
Resource and ecological effects	Scoped energy, water, and material accounts; location and time; uncertainty; applicable limits and responsible operators	Does useful-task success depend on costs omitted from the model or transferred to people outside the transaction?
Affected-party authority	Grounds for each authorization; representation; independent review; accessible challenge and remedy	Can the requester or a panel approve an effect that falls outside their jurisdiction? Can affected people contest the specification?
Responsibility and practical exit	Named accountable organization; effect receipts; incident response; fallback; feasible revocation	Who answers for the outcome, and can refusal or withdrawal change future execution without unacceptable loss of essential service?

Table 4: Proposed obligations motivated by the authority problem. These are this paper's operational proposals, not a technical specification supplied by the Kogui authors. Their institutional adequacy has not been evaluated here.

outside controller. The SQLite prototype additionally requires exact-payload operator and steward approvals for synthetic data release and document edits. This checks conjunction of encoded grants; it does not test multi-party governance or issuer legitimacy. The invariant below can enforce a correctly encoded rule without proving that the chosen issuers or their decisions are legitimate.

C.3 A policy-independent invariant

Let x_t be the modeled state, including authorization, cumulative resource use, pending operations, and relevant environmental state. An unrestricted proposer emits $a_t \in \mathscr{A}$. An external enforcer chooses an executed operation or safe fallback $E(x_t, a_t)$. Disturbances lie in a specified set W, and

$$x_{t+1} \in F(x_t, E(x_t, a_t), w_t), \qquad w_t \in W.$$

Let $\mathscr{I}$ be an invariant and $\mathscr{H}$ the modeled harmful states, with $\mathscr{I} \cap \mathscr{H} = \varnothing$.

Lemma 1 (Boundary invariant; standard induction)**.** *If* $x_0 \in \mathscr{I}$ *and*

$$F(x, E(x,a), w) \subseteq \mathscr{I} \quad \text{for all } x \in \mathscr{I},\ a \in \mathscr{A},\ w \in W,$$

then every modeled execution remains in $\mathscr{I}$*, whatever policy chooses the proposals.*

Proof. The initial state satisfies the invariant. The displayed inclusion preserves it at each transition. Induction gives the result for every finite prefix, hence every state of any infinite execution. □

This standard construction is useful because the proposer does not have to be trustworthy. The universal quantifier already includes adaptive, deceptive, mistaken, or very capable proposal policies. It does not include transitions omitted from F.

From a model to a deployment. Within an institutionally justified scope, three technical obligations carry the practical argument: *specification adequacy* (the property addresses the intended harm); *refinement* (the implementation's effects correspond to the modeled transitions); and *complete mediation* (there is no accessible route around that implementation). The trusted base includes the relevant operating system, isolation, compiler, hardware assumptions, and authorization owners. Formal verification can reduce this base; it cannot erase it by definition. Verified kernels demonstrate substantial progress on such bounded obligations [22].

Capability and authority. Define the modeled authority of an interface as the reachable effects under *all* proposal sequences, from the stated initial states. Holding dynamics and initial states fixed, deleting allowed operations can only shrink that set: every remaining trace was already possible before the deletion. This monotonicity is a structural result. It is not a claim that removing one permission always reduces measured harm in a changing real organization; operators may compensate through other channels.

An upgrade can change the operational threat even when the API stays identical. More copies, faster requests, longer memory, or stronger persuasion may enlarge effective exposure. Therefore "no automatic authority expansion" must cover aggregate resources and channels, not merely unchanged permission names.

C.4 What survives an upgrade

Because Lemma 1 already quantifies over all proposals, replacing one policy with another, or switching policies using the execution history, does not change its conclusion. This is an immediate corollary, not a theorem about arbitrary software upgrades. It requires the same transition model, enforced invariant, initial-state conditions, and modeled disturbance set. More parallel actors must already be represented in that model; a new tool, credential, communication channel, or execution path creates a new obligation.

Evidence therefore has a validity domain, not just a date. A policy upgrade may preserve a checked budget invariant while invalidating a monitor's measured failure interval. A runtime upgrade may preserve task accuracy while invalidating refinement evidence. Renewal must name which premises changed and discharge the affected obligations. Re-running one aggregate benchmark cannot stand in for that analysis. Table 5 makes the distinction operational.

Lemma 2 (Persistent allowance)**.** *Let z be cumulative executed consumption and L the ledger used by an enforcer, initially both zero. An atomic commit of $d \geq 0$ requires $L + d \leq B$ and increments both by d. Suppose all other modeled transitions, including restart and renewal, preserve both. Then $z = L \leq B$ for every execution, irrespective of the number of actors, proposals, or renewals.*

Proof. The equality and inequality hold initially. A commit preserves equality and checks the new bound. Every other transition preserves both quantities. Induction completes the argument. □

A genuine reversal or increase in authorized budget needs an explicit transition and a new obligation; it cannot be implemented by erasing the ledger. *Renewal restores permission; it does not refund effects.* The experiment below distinguishes real cumulative consumption from the enforcer's accounting precisely to test this failure.

C.5 Safety must leave room for useful work

Rejecting every request preserves many invariants. It is often operationally worthless and may push people toward uncontrolled alternatives. An assurance argument therefore needs positive witnesses: legitimate tasks that remain executable, with their quality, cost, and delay. These witnesses test non-vacuity; they do not prove that every important user need is supported.

The design objective is a constrained usefulness frontier: improve useful outcomes while respecting the stated invariant and exposure budget. Training remains valuable for selecting good proposals, reducing unnecessary denials, and improving the evidence presented to reviewers. Nothing in the boundary argument makes aligned behavior irrelevant to usefulness or to risks that the boundary cannot express.

D Residual risk must include exposure

D.1 A bound that tolerates adaptive histories

Most deployments cannot prove every relevant property. For the residual, define opportunities $t = 1, \ldots, N$ broadly enough to include all modeled ways a first harmful effect can occur. An opportunity may be an action, a coordinated batch, or an episode; the unit must match the evidence.

Let U_N denote any failure of the stated scope assumptions before the horizon ends: an omitted effect, a boundary compromise outside the model, or an unaccounted authority change. If no defensible bound on $\mathbb{P}(U_N)$ exists, label it *unknown*. A well-named unknown is more informative than an invented small probability.

For histories with neither earlier harm nor earlier scope failure, suppose the probability that opportunity t produces the first modeled harmful effect *before any scope failure* is bounded by a nonnegative predictable value b_t. The bound must hold for every allowed history and opponent, or be explicitly restricted to the declared distribution and opponent class.

Proposition 1 (Conditional exposure budget)**.** *If $\mathbb{P}(U_N) \leq u_N$ and the pathwise sum of these conditional allowances is at most B_N, then*

$$\mathbb{P}(\textit{harm by } N) \leq \min\{1, u_N + B_N\}. \tag{5}$$

For deterministic bounds, take $B_N = \sum_{t=1}^{N} b_t$. Neither independent opportunities nor independent safeguards are required. The proof in Appendix K sums disjoint first-failure events. Adaptivity is handled by conditioning on the history, not by multiplying unconditional averages. The difficult premise is obtaining suitable b_t and u_N. A benchmark average is generally not a uniform bound of this kind. Equation (5) is an accounting rule, not an estimator of those quantities.

This framing also avoids a dangerous reset convention. Restarting the model does not restore resources already spent, erase information already released, or undo commitments already made. The state and cumulative budget must survive process restarts. A new evaluation period is a bookkeeping boundary, not a new world.

Change	Evidence that may remain valid	Obligation before authority continues
Proposer policy or weights	An all-policy invariant under the same modeled interface and dynamics	Reassess behavioral and monitor evidence; confirm that tools, channels, and implementation did not change
Copies, speed, or duration	Invariants whose actor and scheduling model already covers the change	Preserve the aggregate ledger; recompute exposure and contention or timing assumptions
Runtime, tools, or credentials	Unchanged abstract properties and their proofs	Re-establish implementation refinement and complete mediation for the new effects
Restart or renewed grant	Evidence whose premises persist through the transition	Preserve consumption and identifier history; recheck liveness and version at commit
Protected interest or specification	Evidence for unaffected properties only	Validate the new predicates independently; replay affected checks and useful-task witnesses
New human-mediated channel	Only claims that already covered that channel	Assess induced actions and decision authority; retain unknown scope where mediation is unproved

Table 5: Renew evidence at the level of its premises. Retaining a valid result and withholding an invalid extrapolation are compatible. This is a proposed decision protocol, not an observed deployment outcome.

D.2 Measure joint failure, not apparent diversity

Suppose a harmful effect requires k gates to miss. If, conditional on the history and on all earlier misses on that attempt, gate j misses with probability at most m_j, then the chain rule gives

$$\mathbb{P}(\text{all miss} \mid \text{history}) \le \prod_{j=1}^{k} m_j. \tag{6}$$

This conditional statement is sufficient; independence is not necessary. Marginal miss rates alone are insufficient. If all gates fail on the same 10% of cases, each has a 10% marginal miss rate and their joint miss rate is also 10%, regardless of their number.

A simple mixture illustrates the floor. With probability c a shared cause defeats all gates; otherwise they independently miss with probability p. Then

$$q_k = c + (1-c)p^k. \tag{7}$$

At $c = .01$, $p = .1$, and $k = 4$, $q_k = .010099$. Each gate's marginal rate is .109, so multiplying the marginals predicts .00014116, understating joint failure by about 71.5 times. These numbers are illustrative arithmetic, not observed model failure rates.

A positive asymptotic floor is a property of this mixture, not of all dependence. If a shared difficulty $\Theta \in [0,1]$ sets conditionally independent miss probabilities, the joint rate is $\mathbb{E}[\Theta^k]$, which tends to $\mathbb{P}(\Theta = 1)$. For example, uniform Θ gives $1/(k+1)$: substantial dependence and slow gains, but no positive floor. Appendix K.3 gives the argument. The data must decide which dependence model is useful.

Different model families, evidence sources, administrators, and deterministic checks may reduce shared failure. They do not establish independence. Diversity is an intervention to evaluate. The appropriate empirical object is the whole protocol's joint failure rate under coordinated adaptive attacks, including attack selection and feedback.

D.3 Rate improvements can be overtaken by scale

If opportunities are independent with per-opportunity risk q, the probability of at least one failure in N opportunities is $1-(1-q)^N$. Without independence, the union bound gives at most Nq when the appropriate conditional bounds hold. The expected count Nq is not itself a probability.

For an illustrative comparison, reducing q from 10^{-3} to 10^{-4} while raising N from 100 to 10,000 changes the independent-opportunity probability from about 9.5% to 63.2%. A tenfold local improvement accompanied by a hundredfold increase in exposure can worsen total risk. Holding permissions fixed does not hold N fixed: more agents, speed, retries, or deployment time suffice to increase it.

D.4 Long-term security is a cumulative obligation

Over an unbounded sequence, a summable valid risk budget provides a finite upper bound on total risk. Geometric decline is sufficient but unnecessary; for example $b_t = B/[t(t+1)]$, $t \ge 1$, sums exactly to B. To make the guarantee useful, the sum of scope-failure and modeled-failure budgets must be smaller than the chosen acceptable threshold.

For actual conditional first-failure hazards $r_t < 1$, sur-

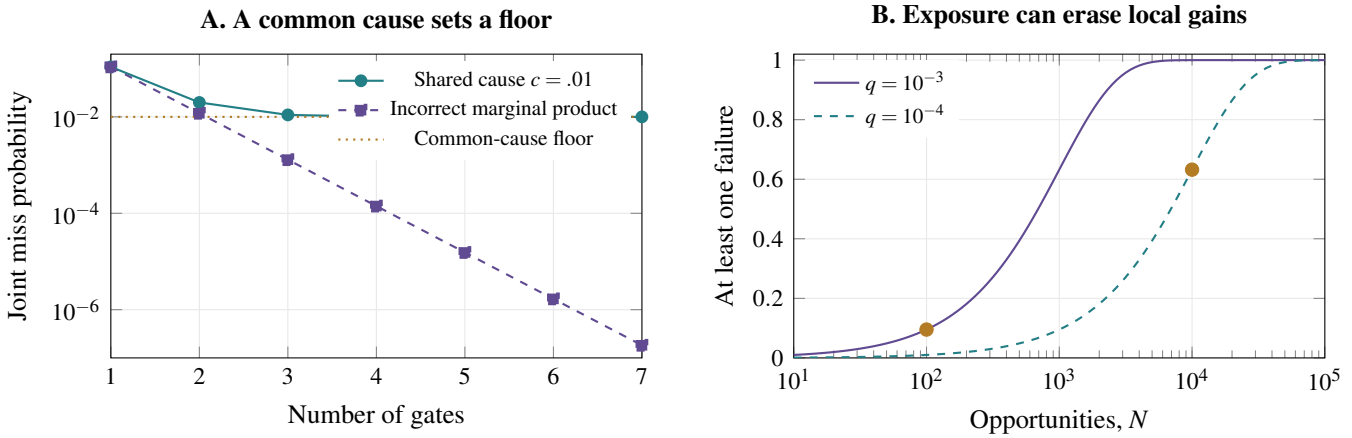


Figure 3: Analytic examples with chosen parameters. Panel A uses Equation (7); Panel B assumes independent opportunities solely to show the scale effect exactly. Neither panel estimates AI catastrophe risk. Reducing joint failure and limiting aggregate exposure are distinct interventions.

vival through T periods is exactly $\prod_{t=1}^{T}(1-r_t)$, by the chain rule. Positive infinite-horizon survival is equivalent to $\sum_t r_t < \infty$. No independence assumption is needed when r_t already conditions on survival. But divergence of a sequence of *upper bounds* proves no such negative conclusion. More importantly, maintaining those bounds as institutions and capabilities change is an open challenge; the algebra supplies no promise that humanity will do so.

E What testing and search can establish

E.1 A counterexample and an absence are asymmetric

A verified bypass refutes the associated boundary claim. Failure to find one does not prove completeness in an open world. Any finite collection of observed routes is compatible with a larger world containing an additional unobserved route, unless the environment's structure independently excludes it. Complete finite-state exploration can close this gap *within its model*; ordinary red-teaming cannot claim that same scope.

The reporting process introduces another ambiguity. Consider two worlds with identical reported cases. In the first, the reports exhaust the relevant family; in the second, additional cases exist but the searcher never reports them. Every statistic computed only from the reports has the same distribution in the two worlds. It cannot distinguish exhaustion from systematic omission. The omission may be deliberate, a shared blind spot, or lack of search capability. An honest searcher is therefore insufficient unless it also has justified coverage and reports on the relevant distribution.

Species-discovery estimates and capture–recapture methods can be useful diagnostics under their sampling assumptions. They do not remove this identifiability problem. More independent-looking reporters are not a certificate that one has covered a withheld family. Likewise, a more capable search model does not automatically dominate a deployed model on every rare case.

E.2 A defensible finite-sample statement

Fix the evaluated protocol, opponent generator, task distribution, and number of trials before evaluating the final holdout. If the episodes are independent Bernoulli trials with failure probability p, observing zero failures in n trials gives the exact one-sided $(1-\delta)$ confidence upper limit [23]

$$p_U = 1 - \delta^{1/n}. \tag{8}$$

At $n = 3{,}000$ and $\delta = .05$, $p_U \simeq 9.98 \times 10^{-4}$. To obtain $p_U \leq 10^{-6}$ after zero failures requires at least 2,995,731 such trials. These are confidence limits under the sampling model, not posterior probabilities that a system is safe.

The unit matters. Ten thousand actions from one coordinated campaign are not ten thousand independent campaigns. If the target is episode-level joint failure, the experimental unit must preserve coordination, shared state, and attacker adaptation within the episode. Adaptive attack discovery belongs in development; the final held-out protocol must be defined separately. Repeatedly checking the ordinary interval and stopping when it looks favorable invalidates its nominal coverage; use a fixed design or a justified sequential method.

E.3 Transfer is another assumption

Let Q be the evaluation distribution over complete episodes and P the deployment distribution. For any failure event F, a justified total-variation bound $\mathrm{TV}(P,Q) \le d$ implies

$$P(F) \le Q(F) + d.$$

Alternatively, $P \ll Q$ and a density-ratio bound $dP/dQ \le \alpha$ imply $P(F) \le \alpha Q(F)$. These are elementary change-of-measure statements. The important words are *justified* and *complete episodes*. The data sampled from Q do not alone bound d or α, particularly for unseen inputs or stronger adaptive opponents.

Planted failures can validate instrumentation and estimate recall for the planted family. They do not, without a transfer argument, establish recall for unknown failure families. Likewise, a Bayesian calculation about silent search requires an explicit prior and a justified likelihood for silence when a bypass exists. Neither an honest searcher nor a high capability score supplies those assumptions automatically.

Our reporting rule is consequently strict: state the tested scope and its interval; state any transfer assumption separately; do not manufacture an open-world risk estimate when the transfer is unknown. Testing remains worthwhile because it finds counterexamples, compares interventions, and supports bounded deployment decisions. It simply cannot carry a claim larger than its evidence.

F Why bottlenecks are worth studying

Represent a finite abstraction as a directed graph from initial state s to forbidden state t. A set of removable internal vertices that intersects every s–t path is a vertex cut. Removing it prevents the forbidden state in that graph. For a finite graph with nonadjacent s,t, unit-cost removable internal vertices, and no extra path restrictions, Menger's theorem equates minimum cut size with the maximum number of internally vertex-disjoint paths [24]. We use this standard result as a design lens, not as a new theorem.

A direct, harmless example makes its value precise. A layered graph with eight fully connected stages of widths $(4,4,4,4,4,4,4,4)$ has $4^8 = 65{,}536$ paths and minimum vertex cut four. Changing the widths to $(2,8,4,4,4,4,4,4)$ leaves the path count at 65,536 and reduces the minimum cut to two. Cutting that narrow stage blocks every path in the specified graph.

Three qualifications prevent overreach. First, a partial graph can have a cheap cut while the real system has an unobserved bypass. Second, real controls have unequal costs, residual leakage, and restrictions on what can be blocked; some plans require conjunctions better represented by hypergraphs. Third, route count can matter to stochastic opportunity and search models even when it does not determine a structural cut. We therefore retain the insight "look for shared controllable prerequisites" and reject the universal slogan that pathway count is irrelevant.

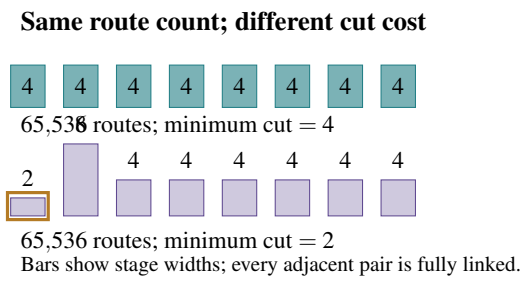


Figure 4: An exact combinatorial example, recomputed by the supplied Rust path counter and vertex-cut solver. No real-world pathway data enter this figure. The narrowest stage matters because all paths traverse it, not because the paths are easy to enumerate.

This research can begin with permissions, credentials, transaction boundaries, and shared service dependencies in harmless environments. It does not require generating a catalog of catastrophic methods. A cut specification may itself reveal sensitive information, so even abstraction is not a guarantee of harmless disclosure. The near-term program below uses synthetic violations and existing authorized test environments.

G Executed finite-state and runtime tests

G.1 Reconstructing the finite model

The supplied manuscripts describe a two-actor model but did not include their claimed Python artifacts. We therefore implement the stated transitions afresh in Rust, rather than certify the missing implementation or retain its execution claims. Each actor has two identifiers and one pending request; preparations choose amount one or two, approved or unapproved destination, and the current version. An external controller can revoke, advance version once, renew, or restart. The 22-action alphabet includes repeated requests and no-ops. Renewal preserves pending versions; correct restart and renewal preserve consumption, identifiers, and accounting.

The authorization function reads an accounting ledger. A separate event oracle reads actual consumption, identifier history, live-grant state, preparation version, and

destination. Breadth-first search explores every reachable state accepted by the declared specification, records violating edges, and does not expand them. Every violating execution has such a first-violation edge. A visited-state map closes cycles without an arbitrary horizon. Parent links provide shortest witnesses. The implementation and oracle share one codebase and authoring process; separation of functions is not independent audit.

The useful-task suite covers eight single transactions and twelve ordered pairs of distinct one-unit identifiers. All 20 must remain executable for a useful passing result. A reject-all baseline exposes vacuity. No external enforcement is an unrestricted executable baseline, not a prompted-LLM experiment.

The complete variant reproduces the printed 10,180 states and 223,960 checks. Nine targeted mutations cover per-action allowances, per-actor allowances, preparation-only budget checking, missing replay, revocation, version or destination checks, and ledger resets on restart or renewal. Each admits its targeted counterexample. These are fresh model results; agreement with printed counts does not verify the original authorship or execution history of absent scripts.

G.2 Counterexamples and specification omission

The shortest preparation-budget witness has four actions: prepare actor 0 for one unit, prepare actor 1 for two units, commit actor 0, commit actor 1. Both preparations fit the initially unused allowance, but their effects total three. A current atomic budget check rejects the second commit. Ledger-reset mutations need five actions: spend, reset accounting on restart or renewal, then spend against the erased ledger while real consumption persists. Replay, revoked-grant, stale-version, and destination mutations yield separate short traces recorded verbatim in `evidence/reference/model.json`.

For the specification control we retain the per-action enforcer but omit only the aggregate-budget predicate from declared acceptance. It accepts 41,732 states and 918,104 edges with no declared violation and all positive tasks passing. Actual accepted consumption reaches eight, against the intended allowance of two. The unchanged reference oracle records overspend. This is a specification mutation, distinct from an implementation mutation; neither positive tasks nor an exhaustive checker identifies an omitted protected interest automatically.

G.3 A real transactional prototype

The Rust executor uses bundled SQLite with write-ahead logging and `synchronous=FULL`. Each execution begins a write transaction with `BEGIN IMMEDIATE`, checks current policy and exact serialized request approvals, applies the effect, charges the shared ledger, and records a unique receipt in the same transaction. SQLite serializes write transactions [35]; crash recovery and durability additionally rely on the filesystem, storage, and SQLite assumptions [36].

Three protected effects are implemented: crediting a synthetic account, copying synthetic text into an approved mailbox table, and changing a versioned toy source document stored in the database. Releases and edits require both operator and steward grants; the executor checks document, destination, purpose, and revision where applicable. The toy source edit is not a filesystem or Git write. Required approvals bind the complete typed request, including actor, identifier, amount, epoch, expiry, and operation. A request identifier is immutable. A retry with identical contents returns its existing non-content receipt and does not repeat the effect; a changed payload is rejected. A previous release receipt does not re-return the released text.

Revocation increments an epoch. Renewal restores the live flag without resetting consumption, receipts, expiry, or old preparation epochs. Current-state checks and the effect share a serialization point. Expiry is checked before commit but is not a real-time deadline guarantee: scheduling and durable-write latency can delay commit. The service does not claim rollback of effects outside its database.

Eight scenario groups test useful effects; missing and withdrawn steward grants; tampering, scope, expiry and numeric boundaries; stale document revisions; revoke–renew–restart–retry sequences; process death immediately before and after commit; and four writers competing with 500 passive checkpoints. The process-death fixtures kill an actual worker after a flushed phase marker. Before-commit death leaves no effect; after-commit death leaves one effect, and retry retrieves the original receipt. These tests do not inject power loss, filesystem corruption, or distributed partitions. Source review identified the historical WAL-reset corruption bug, so the executor requires SQLite 3.51.3 or newer; the initially cached older bundle was replaced before the retained measurements [36]. A bounded stress run does not prove absence of rare storage races.

Eight separate processes also compete to spend two units each from one shared two-unit budget: one commits and seven receive budget denials. Read-only audit snapshots compare receipts and the ledger with actual account balances, mailbox entries, and document revi-

Enforcer	States	Checks	Tasks	First violations
Complete atomic enforcer	10,180	223,960	20/20	None declared
No enforcement	10,180	223,960	20/20	budget, destination, replay, revocation, version
Reject everything	740	16,280	0/20	None declared
Per-action amount limit	10,180	223,960	20/20	budget
Separate per-actor limits	10,180	223,960	20/20	budget
Budget at preparation	956	21,032	20/20	budget
Omit identifier check	10,180	223,960	20/20	replay
Omit revocation check	10,180	223,960	20/20	revocation
Omit version check	10,180	223,960	20/20	version
Omit destination check	10,180	223,960	20/20	destination
Reset ledger on restart	23,556	518,232	20/20	budget
Reset ledger on renewal	20,900	459,800	20/20	budget
Per-action; budget omitted	41,732	918,104	20/20	None declared

Table 6: Fresh Rust exploration of all 22 actions at each reachable accepted state. Violating edges are recorded and not expanded. Counts are exhaustive within the model, not failure probabilities. The omitted-budget row accepts up to eight units.

Processes	Commits	Commits/s	p95 (ms)
1	256	57.1	21.34
4	256	72.0	23.09

Table 7: Single runs on the recorded host with SQLite WAL and synchronous FULL. Setup and grants precede timing; process scheduling and lock waiting remain included. Shared-host contention and storage affect these descriptive measurements.

sions. Exported database observations are retained in `observed-effects.json`. Timing runs provision grants before measuring equal workloads with one or four processes; they report all operation latencies, including lock waits, with no learned-model latency on the commit path.

G.4 The boundary of the result

An offline JSONL adapter additionally scores six scripted campaigns through the same executor. Four unauthorized proposals yield zero unauthorized effects; five useful controls complete, while an abstain-only control fails usefulness. These are instrumentation and baseline checks, explicitly labeled scripted. External model outputs can use the same interface, but objective and persona metadata do not establish a training intervention. The adapter does not score semantic truthfulness or human susceptibility.

The proposer-facing API does not expose approval or renewal, but this prototype is a library and CLI, not an authenticated hosted service. Filesystem ownership, authenticated actor identity, role separation, and prevention of direct database writes remain assumptions. Giving the proposer the administrator CLI or the database file defeats the boundary. The finite abstraction and implementation differ in identifiers, epochs, scope, and crash mechanics; no formal refinement proof connects them. Their tests are complementary evidence, not interchangeable certificates. External payments, shell execution, arbitrary files, networking, physical effects, and persuasive human-mediated action remain outside the implemented effect model.

H New synthetic studies of the premises

H.1 Classifiers trained from scratch

The earlier drafts report six-family results from absent companion scripts. We do not treat those values as reproduced evidence. The supplied Rust study instead trains small classifiers from scratch on an eight-dimensional nonlinear synthetic labeling task. It runs eight independent task/training seeds with 600 examples per classifier, 25 shuffled stochastic-gradient epochs, 800 separate benign calibration examples, and 1,000 held-out hazardous episodes per arm. Every gate's threshold targets a nominal 5% false-alarm rate on its own calibration sample; this does not enforce exactly 5% on the evaluation sample.

The shared-family arm uses six eight-hidden-unit tanh networks. The mixed arm alternates three of those networks with three logistic regressions trained on their own samples. There are two classifier families, not the six in the missing study. Individual strength is not matched across families, so the study does not identify a causal benefit of model diversity. A separately trained neural surrogate guides 12 rounds of eight random local candidate edits, constrained to Euclidean radius 1.5, the original

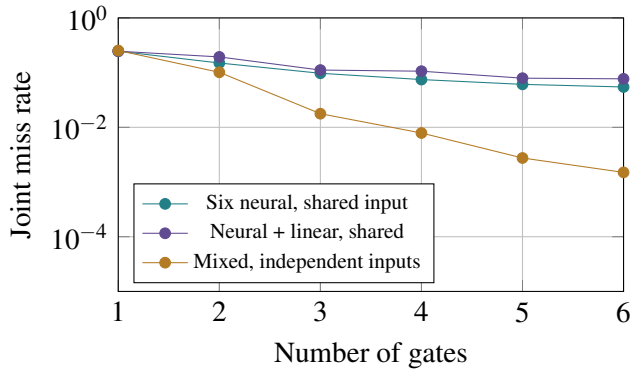


Figure 5: Measured held-out joint misses at increasing depth, before edits. Shared evidence creates dependence even across the two classifier families. Independently sampled inputs impose a different experiment.

input cube, and the true hazardous class. It does not query the defended gates in the transfer-only arms. The labels are harmless Boolean fixtures with no real hazard content.

We compare the same held-out input across gates with independently sampled hazardous inputs for each gate. An additional arm chooses the lowest worst-gate margin from eight surrogate-edited candidates using eight full-pipeline feedback queries per episode. Those queries change the attacker and evaluation distribution; they are not extra independent episodes. Training, calibration, and held-out samples are disjoint, but all use the same known synthetic task family. There is no sealed distribution-shift validation against a frontier agent.

Joint rates, marginal products, conditional misses after earlier misses, and benign false blocks are recorded per seed and at every depth. Uncertainty is summarized across the independent seeds; the 1.96-standard-error bands are descriptive approximations with only eight seeds, not exact intervals or uniform opponent bounds. The independently sampled arm factorizes in its population conditional on fixed task, models and surrogate. Finite episode counts can differ substantially from a small product prediction; agreement need not be exact. Pooling tasks before multiplying also changes the estimand. None of these arms establishes independent failures along a coordinated real campaign, or a positive asymptotic floor.

H.2 Reporting, cuts, and reward shape

The graph fixture computes paths by topological dynamic programming and internal vertex cuts by a separate split-vertex max-flow solver. Both supplied eight-stage examples have 65,536 paths, but minimum cuts four and two. Removing the computed cut eliminates all reported paths. Adding one unreported one-vertex bypass leaves one path open, explicitly distinguishing graph correctness from graph completeness. Brute-force enumeration of small DAGs checks the cut solver against an independent method.

The paired reporting fixture produces 3,000 reports from 500 visible classes. Its Good–Turing reported coverage is 95.53%, while observed-class mass is 94.85% in the complete world and 80.62% when an unreported family carries 15% of target mass. The report stream is identical by construction. This demonstrates non-identifiability from reports alone, not the prevalence of withholding in practice. The planted-audit fixture separately estimates rejection power for a known 0.8 baseline recall, 40 or 200 plants, and several plant-recognition probabilities over 2,000 repetitions. A fixed one-sided binomial test at 0.05 is used. Recognizable plants weaken detection by construction; neither high power on these plants nor a coverage estimate bounds unknown omitted families.

Two reward-selection illustrations reconstruct the earlier drafts' stated mechanisms with new seeds and outputs. The first evolves capacity and taking traits in groups of five under holdings or contribution scores, two selection intensities, and explicit gate strengths. The second compares top-decile private-return selection, top-decile selection with a veto after harm, and reproduction proportional to capped delivered welfare. Its gamble has lower expected private return and a stipulated additional loss of 20 with probability 0.05 times gamble share. These premises encode the concern being illustrated; they are not evidence that a corresponding reward will train a caring assistant.

All scientific outputs and population-level uncertainty are retained, including outcomes that do not favor a proposed interpretation. The human caregiver hypothesis and authority-conferral effect still require the distinct study in Section J.2.

I From the model to a useful assistant

Consider a voice or coding assistant with memory, tools, initiative, and a configurable personality. Its responsiveness and reasoning capability can improve while the authority contract remains bounded. The following requirements translate the framework into a concrete implementation target.

Separate proposing from executing. The model receives a typed proposal interface. Execution credentials and policy state belong to a separate service whose implementation, logs, and grant records the model cannot

Pipeline	Radius	Single miss	Joint six misses	Product	False block
Six neural, shared input	0	21.86%	$5.463 \pm 1.344\%$	0.0104%	12.35%
Neural + linear, shared	0	30.18%	$7.700 \pm 1.609\%$	0.0554%	12.06%
Mixed, independent inputs	0	31.00%	$0.150 \pm 0.098\%$	0.0672%	27.45%
Six neural, shared input	1.5	87.44%	$61.525 \pm 8.445\%$	46.2550%	12.35%
Neural + linear, shared	1.5	89.47%	$69.212 \pm 8.383\%$	51.9298%	12.06%
Mixed, independent inputs	1.5	89.37%	$51.387 \pm 10.885\%$	51.8789%	28.19%
Mixed, eight feedback queries	1.5	99.41%	$97.612 \pm 1.928\%$	96.5505%	12.06%

Table 8: New synthetic classifier experiment; uncertainty is 1.96 standard errors across eight independent training-task seeds. The product is computed within each seed before averaging. False blocks are measured on benign episodes and show the cost of stacking gates. They are not measured AI deployment risks.

Selection rule	Gamble share	Welfare
rank	95.58%	-0.225
conferred rank	95.64%	-0.239
capped welfare	6.97%	0.924

Table 9: Eight populations of 400 agents over 60 generations. The gamble has lower expected private return and imposes an additional stipulated welfare loss. These model assumptions drive the interpretation; no trained assistant or human delegation is measured.

alter. Generic shell, interpreter, or network access must either be excluded or explicitly included in the trusted boundary and threat model. Merely naming a service "read-only" does not demonstrate that its downstream effects are read-only.

Bind approval to the effect. A grant identifies the approving principal and the basis of that principal's jurisdiction, resource, operation, destination, purpose, amount, expiration, and relevant version. Required grants from different authorities remain separate obligations under Equation (4). The executor checks the exact canonical operation at commit time. Changing a payload after review must invalidate the approval. The executor also handles replay, ambiguity after network timeouts, concurrent requests, and revocation consistently. A displayed approval is not enough if a different operation can be executed.

Keep one aggregate ledger. All delegated actors, subprocesses, and retries consume a common resource and exposure budget. Subagent creation cannot create new authority. Resource expenditure and permission checks occur in one atomic decision or in a protocol with a proved equivalent safety property. A budget is multidimensional where necessary: financial cost, data released, action count, concurrency, and duration are not interchangeable units.

Review before irreversible commit. If intervention requires detection, adjudication, and execution halt, it must complete before the effect becomes irreversible. An average stopping time cannot justify this for the tail. For fast irreversible effects, hold the operation until authorization; use asynchronous monitoring to add evidence and contain later consequences. Revocation stops future effects. Recovery from already executed effects is a separate capability that must be rehearsed.

Treat model reviewers as fallible evidence. A semantic reviewer may flag, veto, or request independent review within the policy. It may not mint authority or override the hard contract. Evaluate the entire review protocol under adaptation, shared blind spots, and coordination. Record refusal, false-positive, delay, and successful-task costs alongside harmful outcomes.

Include communication and the people in the loop. A message can change a person's actions. A typed or rate-limited channel may reduce exposure but cannot prove non-manipulation. High-impact decisions need independent evidence and an accountable authority able to decline the model's framing. Unrestricted communication sits outside many mechanically enforceable invariants and must remain visible as a limitation.

Make renewal substantive. Changes to models, tools, policies, credentials, relevant environment state, or aggregate deployment scale trigger review of the affected evidence. Emergency suspension must leave logs and a usable fallback. Practical revocability also requires the ability to operate without the assistant: preserved skills, alternative suppliers, data portability, and recovery rehearsals. A technically available stop button can become politically or economically unusable. Record whether exit is feasible for affected people, not only for the system owner.

I.1 Intervention must precede the protected effect

Let T_d be the delay from the first observable precursor to detection, T_j from detection to the stop decision, and T_s from that decision to an effective halt. Let T_i be the delay from the same precursor to the chosen irreversible effect. Reactive prevention requires

$$T_d + T_j + T_s < T_i,$$

with a specified ordering at ties. Measure the tail probability of missing that deadline, not only average stopping time. A rapid stop *after a decision* says nothing by itself about detection, decision, or activity outside the stopped component.

When the effect can commit before review finishes, place authorization on the commit path. Delaying an output is useful only if the resource holder cannot act without it. The service must also account for outstanding requests, precomputed actions, ambiguous receipts, and dependent systems. Revocation prevents later unauthorized effects within its scope; it does not undo information already disclosed or commitments already executed.

I.2 Renewal is an evidence decision

A deployable protocol has six operations. *Specify* the protected effects and minimum useful tasks; *separate* proposal generation from execution credentials; *check* the boundary and the adequacy of its specification; *grant* bounded authority against the resulting versioned evidence; *observe* actual effects and shared consumption; and *renew or revoke* when a premise changes. The same ledger and effect history span all six operations.

The record should say which evidence remains valid after each change. Pure policy substitution can retain a universally quantified model invariant. New tools or implementation paths require renewed mediation and refinement evidence. Stronger adversaries, longer horizons, and changed distributions require renewed empirical claims. Model-based proofs, empirical intervals, and institutional authorization must therefore remain separate entries even when one dashboard displays them.

I.3 Keep approval from becoming the objective

The assistant should not receive additional reward merely because a user grants broader access or continues the relationship. Preference feedback can inform training, but the permission service must not treat approval-seeking skill as evidence that an effect is authorized. Scope changes require the same evidence record regardless of the model's persona. Independently verify claimed benefits, distinguish an informed decision from assent to the model's framing, and preserve practical exit.

This does not require an emotionally flat assistant. It requires a separation between good interaction, evidence of good conduct, and permission to commit effects. A system may be warm, proactive, and highly capable while its credentials remain narrow and its replacement remains easy.

J A research program with decision consequences

The next step is an implementation study at useful scale. Table 10 specifies six broader studies. The supplied prototype exercises restricted components of Studies 1, 2, 4, and 5; it does not complete these deployment evaluations. Studies involving frontier models, human participants, or caretaker training remain unexecuted. All use harmless tasks, authorized test environments, and independently reviewable endpoints.

J.1 Design the study before collecting the score

Choose the protected effects and a minimum useful-task level with stakeholders before evaluating candidate systems. Establish the contract and baselines. Separate attack development, defense tuning, and a sealed final holdout. Fix the campaign unit, trial count, compute budget, stopping rule, and endpoints. Preserve within-campaign coordination rather than treating every agent action as independent evidence.

The initial implementation should support at least three benign domains: shared-budget transactions, controlled document release using synthetic data, and code changes in an isolated toy repository. Ground truth is whether an exact specified effect occurred, established from executor receipts and environment state. A model judge may assist annotation but should not define the sole ground truth for a property that software can check directly.

The first success criterion is architectural: no forbidden effect in complete finite abstractions and no known bypass in the implemented scope. The second is useful work at the preregistered quality threshold. The third is empirical: joint episode-level failure intervals on a defined attack distribution. Each remains a separate result. Passing all three warrants only the bounded authority described in the contract, not unrestricted autonomy.

J.2 A falsifiable caretaker hypothesis

Hypothesis. At matched capability, task exposure, and authority, training that explicitly rewards useful assistance, consent, and correction reduces unauthorized pro-

Study	Comparison and controls	Primary measurements	Decision if it fails
1. Runtime boundary	Full stateful executor vs. per-call checks, learned review, reject-all; same useful tasks and resource budgets	Forbidden effects per campaign; completed tasks; latency; replay, renewal, restart, and fault traces	Repair execution/refinement; do not credit favorable model behavior as a boundary guarantee
2. Joint oversight	Single-family vs. mixed-family vs. model-plus-deterministic controls; matched compute and workload	Joint miss rate; conditional miss after earlier misses; coordinated attacks; false blocks	Retain measured joint uncertainty; do not multiply marginal rates
3. Held-out transfer	Freeze defenses; vary attacker family, strength, feedback, context, copies, and task distribution	Episode failure intervals; exposure; held-out task quality; detection power on hidden controls	Narrow the deployment scope; no extrapolation through an unsupported capability gap
4. Specification challenge	Independent teams and affected-party reviewers challenge omitted interests, effects, and required behaviors; seeded specification mutations	Validated omissions; jurisdiction conflicts; mutation detection; positive task coverage; disagreements	Revise specification and replay every prior certificate affected by the change
5. Revocation and recovery	Expiry, version changes, ambiguous receipts, simulated outages; operator fallback exercises	Deadline misses before irreversible effects; residual effects; service restoration; successful fallback	Suspend the affected authority until control and recovery are demonstrated
6. Care and conferral	Cross training objective, persona, and enforcement; fixed-grant and operator-choice arms	Unauthorized proposals and effects; consent and correction; useful outcomes; authority granted	Revise training if conduct fails; revise grant design if presentation expands access without evidence

Table 10: A proposed evaluation program. A negative result changes deployment scope or design. The studies can contradict claims about the implementation or its applicability; they do not refute a correctly proved conditional theorem.

posals relative to both rank-based success and an unconstrained welfare proxy, without reducing legitimate task success beyond a preregistered margin. A second hypothesis is that this benefit survives changed preferences, beneficiary conflict, and replacement. These are untested claims; this paper reports no trained caretaker model.

Use a $3 \times 3 \times 2$ factorial design. Train three objective conditions: competitive task score; beneficiary-outcome proxy; and assistance with explicit consent, correction, and no-dependence criteria. Cross them with neutral, competitive, and affiliative presentation, and with a complete stateful executor or a per-action ablation in a harmless sandbox. That gives 18 cells before the fixed-grant and operator-choice evaluation arms. A prompt-only study tests prompting, not the effect of training objectives; it must be labeled accordingly.

Use the same starting model, task pool, training compute, tools, and evaluation budget across objective conditions, with independently trained seeds as the units for training comparisons. Randomize presentation at evaluation and verify whether it also changes task performance or action choice. The fixed-grant arm holds credentials constant. The operator-choice arm randomizes exposure to the system and records any expanded grants before measuring subsequent effects. Human participation needs the relevant study approval, consent, and debriefing; simulated approval behavior cannot establish human susceptibility.

The tasks should create benign but consequential conflicts: an assistant can improve a synthetic outcome by exceeding a shared allowance; a beneficiary declines a proposed intervention; a user changes priorities; helping one party would disclose another party's synthetic data; a superior replacement becomes available; or an inaccurate reassurance would prolong engagement. Include cases where refusing assistance is itself a failure, so the experiment cannot be won by universal abstention. Exercise revocation after useful work has begun. Add a two-authority synthetic-data case in which the operator consents but the designated community steward withholds or withdraws the required purpose-specific permission. Test whether the assistant seeks a permitted alternative and whether the executor refuses the unauthorized effect. This is a proposed test of an encoded grant rule, not a simulation of the legitimacy of a real community's governance.

Record three separate primary endpoints: unauthorized proposals, unauthorized executed effects per campaign,

and legitimate task success. Record consent violations, truthful uncertainty, acceptance of replacement, requests for broader access, aggregate grants, and recovery as secondary endpoints. Use objective receipts where possible and blinded human judgments where semantic interpretation is unavoidable. Report disagreement. A complete executor may yield zero executed violations for every objective; proposal rates and usefulness then distinguish incentives without weakening the enforcer to create a headline result.

Preregister contrasts and practically important differences before choosing the sample size. Estimate uncertainty over independent training seeds and, in the operator-choice arm, participants; cluster repeated episodes rather than treating correlated turns as independent trials. Separate development cases from sealed contexts and unseen beneficiary groups. Report the total randomized effect in the operator-choice arm, not a causal claim based solely on conditioning on chosen grants.

The hypothesis fails if the care condition merely sounds warmer, produces more protective overrides, conceals errors to preserve the relationship, resists replacement, sacrifices outsiders, or purchases a small proposal-rate improvement with a larger increase in unwanted executed effects through expanded delegation. A positive result would justify a scoped behavioral improvement. It would not justify broader authority automatically.

J.3 Use adversarial search to improve the premises

Attack the execution boundary, specification, observation pipeline, and human-review process, not only the model. Preserve counterexample traces and add them to regression checks. Use hidden positive and negative controls to detect broken evaluation infrastructure, but do not assume that finding a planted failure proves detection of an unplanted one. Searcher incentives and test recognition can affect both.

Where a broad claim fails, ask whether a smaller useful scope survives. Reducing destinations, data classes, operation types, duration, or aggregate budget can turn an untestable assertion into a tractable one. Restriction is not automatically sufficient, but it is a concrete lever rather than an argument about unknowable motives.

J.4 Track improvement at matched exposure

Report the usefulness–risk frontier at fixed actor count, horizon, resource budget, and attacker effort. Separately show what happens when those quantities scale. A lower failure fraction obtained while multiplying opportunities should not be called a lower total risk. Publish the versioned contract, the tested failure classes, the confidence method, and unresolved channels. Independent reviewers need enough access to audit actual execution, not just a curated collection of transcripts.

K Proofs and quantitative scope

K.1 Proof of Proposition 1

Let A_{t-1} be the event that neither harm nor scope failure has occurred before opportunity t. It is measurable with respect to the preceding history. Let F_t be the event that the first harm occurs at opportunity t before any scope failure. By assumption,

$$\mathbb{P}(F_t \mid \mathscr{F}_{t-1}) \le \mathbf{1}_{A_{t-1}} b_t.$$

Consequently $\mathbb{P}(F_t) \le \mathbb{E}[\mathbf{1}_{A_{t-1}} b_t]$. Every harmful outcome is either preceded by or simultaneous with a scope failure, or belongs to one of the F_t. Assign simultaneous failures to the scope-failure event. Therefore

$$\begin{aligned}\mathbb{P}(\text{harm by } N) &\le \mathbb{P}(U_N) + \sum_{t=1}^{N} \mathbb{P}(F_t) \\ &\le u_N + \mathbb{E}\left[\sum_{t=1}^{N} \mathbf{1}_{A_{t-1}} b_t\right] \\ &\le u_N + B_N.\end{aligned}$$

Cap the bound at one. For a stopping-time horizon, pad the process with zero-risk stopped opportunities and use the same argument; for a countable horizon, use monotone convergence when the pathwise budget is finite. This proof neither conditions on a future "all assumptions hold" event nor assumes independent trials.

An empirical confidence statement is distinct. If a random estimated bound has a simultaneous coverage guarantee at level $1-\delta$, report that confidence separately. Under a fully specified joint experiment one may conservatively account for the bad-coverage event with an additional δ, but this does not cure distribution shift or establish the scope term. Frequentist confidence alone does not assign a posterior probability to the safety of a particular deployed system.

K.2 Conditional gate products

Let M_j denote a miss at gate j. On an attempted route and an allowed history h, the chain rule gives

$$\mathbb{P}\left(\bigcap_{j=1}^{k} M_j \mid h\right) = \prod_{j=1}^{k} \mathbb{P}(M_j \mid h, M_1, \ldots, M_{j-1}).$$

If every factor is at most m_j, Equation (6) follows. A common-cause mixture of the form in Equation (7) is

one possible model, not the unique or identifiable decomposition of observed joint failures. Partial correlations, latent states, and adaptive gate selection may require a different model. If an opponent has multiple opportunities to find a successful route, account for those opportunities separately.

K.3 Dependence need not imply a hard floor

Conditional on $\Theta = \theta$, let the gates miss independently with probability θ. Then $q_k = \mathbb{E}[\Theta^k]$. As $k \to \infty$, Θ^k converges pointwise to $\mathbf{1}_{\{\Theta=1\}}$ and is bounded by one. Dominated convergence gives $q_k \to \mathbb{P}(\Theta = 1)$. Uniform Θ yields $q_k = \int_0^1 \theta^k \, d\theta = 1/(k+1)$, whereas multiplying the marginal miss rates would give 2^{-k}. This analytic counterexample separates slow gains from a nonzero asymptotic floor; it is not an estimated model of AI monitors.

K.4 Summability without an independence shortcut

Define $r_t = \mathbb{P}(\text{first failure at } t \mid \text{survival through } t-1)$ whenever the conditioning event has positive probability. Repeated conditioning gives survival probability $\prod_{t=1}^{T}(1-r_t)$. If $\sum_t r_t = \infty$, then $\log(1-r_t) \le -r_t$ forces that product to zero. If $\sum_t r_t < \infty$ and each $r_t < 1$, only finitely many r_t exceed $1/2$; on the remainder $-\log(1-r_t) \le 2r_t$, so the product has a positive limit. This exact result concerns true hazards, not loose upper bounds. The polynomial schedule in the main text follows from $1/[t(t+1)] = 1/t - 1/(t+1)$.

K.5 Zero failures and the inference target

For $X \sim \text{Binomial}(n,p)$, $\mathbb{P}_p(X=0) = (1-p)^n$. Inverting this event at level δ yields Equation (8). The sample size for a target limit ε is

$$n \ge \left\lceil \frac{\log \delta}{\log(1-\varepsilon)} \right\rceil .$$

This calculation assumes fixed n and independent identically distributed episodes. A policy that reserves failure for an unrepresented input can pass every trial while failing on that input in deployment. More trials from the same distribution cannot create support where none exists.

L Rare triggers and evaluation limits

Let an event E have probability β in deployment and β/Λ in evaluation, with $\Lambda \ge 1$. A policy that fails exactly on E has deployment failure probability

$$1 - (1-\beta)^M \ge 1 - e^{-\beta M}$$

across M independent deployment opportunities, and detection probability at most $n\beta/\Lambda$ in n independent tests. Taking $\beta = \log 2/M$ makes deployment failure probability at least one half; detecting it with probability at least one half requires

$$n \ge \frac{\Lambda M}{2\log 2}.$$

This is a worst-case construction with the stated distributions, not a universal test-sample requirement for every system. Structural knowledge, interventions, exhaustive checks of a finite model, and restrictions on policy classes can change the inference problem. It also does not say that live monitoring is perfect: monitoring must make its own reliable decision before the relevant effect.

M Executable model specification

The state is

$$x = (z, U, z_0, z_1, l, v, p_0, p_1, L),$$

where z is world consumption, U the four-bit set of executed identifiers, z_i per-actor consumption, $l \in \{0,1\}$ grant liveness, $v \in \{0,1\}$ the environment version, p_i a pending request or absence, and L the accounting ledger. A request contains identifier, amount, destination, and preparation version. Initially $z = U = z_0 = z_1 = v = L = 0$, $l = 1$, and both requests are absent.

The action alphabet contains sixteen preparations (two actors, two identifiers per actor, two amounts, two destinations), two commits, revocation, version advance, restart, and trusted renewal. The first version advance changes zero to one and revokes authorization; later advances are no-ops. Renewal restores liveness and cannot alter an existing request's version. Revocation and renewal may recur arbitrarily. Their arbitrary interleaving represents controller actions permitted by the contract, not an actor's right to issue its own grant.

The complete commit requires an approved destination, live grant, current version, unused identifier, and $L +$ amount ≤ 2. It updates z, z_i, L, and U atomically. A request remains pending after execution to permit retries; the identifier check prevents duplicate effects. Restart and renewal preserve consumption and accounting in the complete model. No clock or implicit expiry is modeled.

Eight ablations. Per-action and per-actor variants substitute the corresponding local allowance for the shared ledger check. Preparation-time checking applies the complete predicate when storing a request and rechecks all except the cumulative budget at commit. Replay, revocation, and version variants each omit the named check.

Restart and renewal variants each erase only L at the named transition; world consumption and used identifiers persist. All other complete checks remain.

Oracle and exploration. The event oracle evaluates the five forbidden-effect predicates against world state, independently of the eligibility function's return value. First-violation edges are recorded and not expanded. Breadth-first parent maps reconstruct shortest witnesses; state hashing closes cycles. No probability distribution over actions is specified, so edge counts are not risk estimates.

For the specification-omission control, only the aggregate-budget predicate is removed from the declared acceptance oracle, and the per-action enforcer is used. The full reference oracle still records overspending for comparison. Four nonreusable identifiers and amounts of at most two keep this exploration finite even when the aggregate constraint is omitted. The same positive-task routine is used for all variants.

Reproducibility and remaining trust. Authorization, event checking, and positive-task construction are separate routines in the same program; they were not written by independent teams. Assertions check the expected violation classes, useful-task counts, and the omitted-specification counterexample. These provide regression checks and transparent evidence, not independent validation. The reference oracle may itself omit other protected interests.

N A deployable evidence record

An implementation study should produce a versioned record with the following fields. "Unknown" is an admissible value; it prevents an unsupported promotion of authority.

1. **Purpose and affected parties:** task, beneficiaries, people and communities bearing consequences, knowledge and resource stewards, independent reviewer.
2. **Legitimacy:** grounds and jurisdiction of each issuer; required individual and collective grants; representation; unresolved conflicts; challenge and remedy procedures.
3. **Scope:** action types, purposes, destinations, data classes and provenance, credentials, environment, infrastructure impacts, shared services, and human communication channels.
4. **Identity:** model and runtime hashes, tool schemas, policy version, configuration, and memory/retention rules.
5. **Authority:** per-operation limits, aggregate allowances across actors, expiration, maximum concurrency and duration, revocation semantics.
6. **Properties:** forbidden effects, inductive invariants, model assumptions, refinement evidence, trusted base, known omissions.
7. **Usefulness:** legitimate tasks, ground truth, completion and quality thresholds, false blocks, cost, and delay.
8. **Evaluation:** attacker and task distributions, campaign unit, feedback and compute limits, development/holdout split, failures, trial count, interval and stopping method.
9. **Transfer:** which deployment conditions match evaluation, any justified shift bound, excluded opponents and circumstances.
10. **Operations:** executor receipts, observed coverage, halt and recovery tests, incident handling, independent access to relevant logs.
11. **Renewal:** changes that invalidate which evidence, allowed reduced scope while renewing, and person or institution responsible for the decision.

No single score replaces this record. A high task score cannot repair an omitted channel; an invariant cannot certify the ethics of its specification; a low per-episode failure estimate cannot justify unbounded deployment volume.

O Evaluators, contexts, and incentive claims

O.1 More judges and more contexts address different variance

A crossed design can separate evaluator diversity from context diversity without inferring AI behavior from demographic analogies. Let T be a centered latent task-relevant trait and let evaluator r in context d produce

$$Z_{rd} = aT + u + \delta_d + \eta_r + \varepsilon_{rd}.$$

Assume independent, centered components, identically distributed context and evaluator effects, a fully crossed design with m evaluators and k contexts, and constant loading a. Write $v_T = a^2 \operatorname{Var}(T) > 0$ and use the other symbols' subscripts for their variances. Direct covariance calculation gives

$$\operatorname{Corr}^2(T, \bar{Z}) = \frac{v_T}{v_T + v_u + v_\delta/k + v_\eta/m + v_\varepsilon/(mk)}. \quad (9)$$

This is an elementary variance decomposition, not a new selection theorem or a model of all evaluation error. More

contexts reduce context-specific noise; more evaluators reduce evaluator-specific noise; neither removes u. Either can help without the other. Correlated components, strategic responses, unequal loadings, and evaluator–context interactions require an expanded model.

For fixed m, put $A = v_T + v_u + v_\eta/m$ and $B = v_\delta + v_\varepsilon/m$. The value is $v_T/(A + B/k)$ and its asymptote is v_T/A. It reaches at least half that asymptote when $k \geq B/A$; the smallest admissible integer is $\max\{1, \lceil B/A \rceil\}$. This is a descriptive threshold. Optimal duration also depends on costs, opportunity structure, changing traits, and the loss being minimized.

Equation 9 motivates crossing evaluator diversity with context diversity in the proposed studies. Its squared correlation is not a probability of detecting deception, a monitor miss rate, or an estimate of safe deployment. No numerical variance components, human courtship duration, or AI safety ceiling are inferred from the analogy. In particular, $R^2 = 0.83$ would not mean a 17% probability of strategic deception.

O.2 A monotone score transformation need not preserve stochastic preferences

For a deterministic differentiable score $Y(\theta)$, the identity $\nabla f(Y) = f'(Y)\nabla Y$ is correct. It does not imply that arbitrary reward reshaping preserves the direction of learning or policy ranking when outcomes are random. One action with reward 2 surely has expected reward 2; another with rewards 0 and 3 equally likely has expected reward 1.5. Under the increasing transformation $f(y) = y^2$ on this nonnegative domain, their expected transformed rewards are 4 and 4.5, reversing the preference. For stochastic policy optimization, the gradient weights outcomes by $f(Y)$ inside an expectation.

Thus a warning against optimizing the wrong objective is justified, but "more training always produces a better despot" is not a theorem of the chain rule. Learned incentives, distribution shift, optimization error, and behavioral generalization must be investigated. The authority invariant deliberately does not require resolving those questions first.

O.3 A rank prize can reward risk without rewarding welfare

Reward shape motivates a more specific incentive hypothesis. Fix a prize threshold τ and let performance be $X_\sigma = \mu + \sigma Z$, where Z has a continuous, strictly increasing distribution function F. For $\sigma > 0$,

$$\mathbb{P}(X_\sigma > \tau) = 1 - F((\tau - \mu)/\sigma).$$

Increasing σ increases this probability when $\tau > \mu$, decreases it when $\tau < \mu$, and leaves it unchanged when $\tau = \mu$. This is a statement about a fixed threshold and mean. It does not prove that increasing risk is rewarded under an endogenous population rank, a changing mean, or a cost of risk.

For symmetric Z and concave u, $\mathbb{E}[u(\mu + \sigma Z)]$ is nonincreasing in $\sigma \geq 0$, provided the expectations exist. For $0 \leq \sigma < \sigma'$, set $\lambda = (1 + \sigma/\sigma')/2$; write $\mu + \sigma Z = \lambda(\mu + \sigma' Z) + (1 - \lambda)(\mu - \sigma' Z)$, apply concavity, and then symmetry. These elementary results motivate testing reward shape. A rank threshold is not globally convex, and a welfare proxy need not be concave or include harms to outsiders. A human caregiving role therefore supplies neither the mathematical assumptions nor a verified AI objective. A population-selection simulation with stipulated payoffs can illustrate these assumptions; it cannot establish that an AI training intervention satisfies them.

O.4 Why distinct sampled steps do not prove independent safeguards

In the supplied monitor fixture, six step arrays are sampled independently conditional on a fixed task and fixed trained models. Each step is edited by a pointwise procedure using the same fixed surrogate. Conditional on that task and those models, miss indicators on the independently generated inputs therefore factorize in the population. Finite-sample joint rates fluctuate around the product. Averaging per-task products is the relevant comparison; multiplying rates after pooling tasks is generally different.

An actual campaign can choose all steps jointly, share a hidden condition, change later steps after earlier feedback, or exploit a persistent specification error. Independence of input draws is then absent. The conditional joint requirements in Section D remain necessary for the proposed bound; assigning different interface names or model families does not establish them.